

Approximation Algorithms with Python

A Computational Companion to Vazirani's Classic Text

Based on *Approximation Algorithms* by Vijay V. Vazirani (Springer, 2001)

Pure Python Implementations — Zero Dependencies

July 24, 2026

Preface

“Although this may seem a paradox, all exact science is dominated by the idea of approximation.”

— Bertrand Russell

This book is a computational companion to Vijay Vazirani’s seminal text *Approximation Algorithms* (Springer, 2001). Vazirani’s book provides the mathematical foundations — theorems, proofs, and algorithmic frameworks. This companion provides **runnable Python implementations** of the core algorithms, along with intuitive explanations that precede the formalism.

Why this book exists:

- Theory and practice belong together. Understanding an approximation algorithm deeply requires implementing it.
- This companion covers the **fundamental algorithms** from Vazirani’s Parts I and II, with chapter numbers matching the original book exactly.
- Pure Python, zero dependencies. Run anywhere, modify freely, learn by doing.

Structure of each chapter:

1. **Intuition First** — Why does this work? What’s the key idea?
2. **Problem Definition** — Formal statement with notation
3. **Algorithm** — Pseudocode and Python implementation
4. **Analysis** — Approximation factor proof sketch
5. **Other Approaches** — What else works? Why this one?
6. **Why Approximation?** — Why can’t we solve exactly?
7. **Applications** — Real-world uses
8. **Tight Examples** — Instances where the bound is achieved
9. **Exercises** — For deeper understanding

Prerequisites: Basic algorithms (sorting, graphs, DP), NP-completeness, and Python familiarity. Mathematical maturity at the level of an undergraduate algorithms course.

Acknowledgments: This work stands on the shoulders of Vazirani’s masterpiece. The PAAL library (paal.mimuw.edu.pl) provided C++ reference implementations for verification.

Contents

Preface	i
1 Introduction	1
1.1 The Approximation Paradigm	1
1.1.1 Vertex Cover: A Gentle Beginning	1
1.1.2 Lower Bound via Maximal Matching	1
1.1.3 The 2-Approximation Algorithm	2
1.1.4 Why Approximation? Why Not Exact?	2
1.1.5 Other Approaches and Their Limits	2
1.1.6 Python Implementation	2
1.2 Applications	3
1.2.1 Tight Example	3
1.2.2 Exercises	3
2 Set Cover	5
2.1 Intuition: The Greedy Instinct	5
2.2 Problem Definition	5
2.3 The Greedy Algorithm	5
2.3.1 Layering: Another 2-Approximation for Weighted Vertex Cover	6
2.3.2 Application to Shortest Superstring	6
2.4 Other Algorithms for Set Cover	6
2.5 Python Implementation	6
2.6 Applications	7
2.7 Tight Example	8
2.8 Exercises	8
3 Steiner Tree and TSP	9
3.1 Intuition: Metric Closure and MST	9
3.2 Metric Steiner Tree	9
3.3 Metric TSP	9
3.3.1 Factor-2 Algorithm (Double Tree)	10
3.3.2 Christofides’ 3/2-Approximation	10
3.3.3 Other Approaches for TSP	10
3.4 Python Implementation	11

3.5	Applications	11
3.6	Tight Examples	12
3.7	Exercises	12
4	Multiway Cut and k-Cut	13
4.1	Intuition: Isolating Cuts	13
4.2	Multiway Cut	13
4.2.1	Minimum k-Cut	13
4.2.2	Other Approaches	14
4.3	Python Implementation	14
4.4	Applications	14
4.5	Tight Example	15
4.6	Exercises	15
5	k-Center	16
5.1	Intuition: Parametric Pruning	16
5.2	Problem Definition	16
5.2.1	Parametric Pruning Algorithm	16
5.2.2	Weighted k -Center	17
5.2.3	Other Approaches	17
5.3	Python Implementation	17
5.4	Applications	17
5.5	Tight Example: Wheel Graph	18
5.6	Exercises	18
6	Feedback Vertex Set	19
6.1	Intuition: Peeling and Layering	19
6.2	Problem Definition	19
6.3	Algorithm and Python Implementation	19
6.4	Analysis: Local Ratio and Cyclomatic Weights	21
6.5	Applications	22
6.6	Tight Example: Bipartite Graph	22
6.7	Exercises	22
7	Shortest Superstring	23
7.1	Intuition: Overlaps and Cycles	23
7.2	Problem Definition	23
7.3	Algorithm and Python Implementation	23
7.4	Analysis: Cycle Covers as Lower Bounds	24
7.5	Applications	24
7.6	Tight Example: Greedy Overlap Heuristic	25
7.7	Exercises	25
8	Knapsack and FPTAS	26
8.1	Intuition: Scaling Values	26
8.2	Problem Definition	26
8.2.1	Exact DP (Pseudo-polynomial)	26
8.2.2	FPTAS via Value Scaling	26

8.2.3	Other Approaches	27
8.3	Python Implementation	27
8.4	Applications	28
8.5	Exercises	28
9	Bin Packing	29
9.1	Intuition: Fitting and Grouping	29
9.2	Problem Definition	29
9.3	Algorithm and Python Implementation	29
9.4	Analysis: Linear Grouping and APTAS	30
9.5	Applications	31
9.6	Tight Example: Next-Fit Heuristic	31
9.7	Exercises	31
10	Minimum Makespan Scheduling	32
10.1	Intuition: Load Balancing and Dual Approximation	32
10.2	Problem Definition	32
10.3	Algorithm and Python Implementation	32
10.4	Analysis: LPT and Hochbaum-Shmoys PTAS	33
10.5	Applications	33
10.6	Tight Example: LPT Heuristic	33
10.7	Exercises	34
11	Euclidean TSP	35
11.1	Intuition: Geometric Dissection and Portals	35
11.2	Problem Definition	35
11.3	Algorithm and Python Implementation	35
11.4	Analysis: Arora's PTAS and Portals	36
11.5	Applications	36
11.6	Tight Example: Shifting Bottleneck	37
11.7	Exercises	37
12	Introduction to LP-Duality	38
12.1	Intuition: Two Sides of the Same Coin	38
12.2	The LP-Duality Theorem	38
12.3	Min-Max Relations as LP Duality	38
12.4	Two Fundamental Algorithm Design Techniques	39
12.4.1	Integrality Gap	39
12.5	Python: Simple Simplex Solver	39
12.6	Exercises	40
13	Set Cover via Dual Fitting	41
13.1	Intuition: Prices as Dual Variables	41
13.2	Dual of Set Cover LP	41
13.2.1	Dual-Fitting Analysis of Greedy	41
13.2.2	Can the Guarantee Be Improved?	41
13.3	Generalizations	41
13.3.1	Constrained Set Multicover	41

13.3.2 Dual Fitting for Multicover	41
13.4 Other Algorithms for Set Cover	42
13.5 Python Implementation	42
13.6 Exercises	43
14 Rounding Applied to Set Cover	44
14.1 Intuition: Threshold Rounding	44
14.2 LP Relaxation for Set Cover	44
14.2.1 Vertex Cover via LP Rounding (Factor 2)	44
14.2.2 Integrality Gap	45
14.3 Python Implementation	45
14.4 Exercises	45
15 Weighted Vertex Cover via Primal-Dual	46
15.1 Intuition: The Classical Schema	46
15.2 Problem Definition	46
15.3 Algorithm and Python Implementation	46
15.4 Analysis: Factor 2 Approximation	47
15.5 Applications	47
15.6 Tight Example: Star Graph and Complete Bipartite Graph	48
15.7 Exercises	48
16 Randomized Rounding	49
16.1 Intuition: LP Probabilistic Rounding	49
16.2 Problem Definition	49
16.3 Algorithm and Python Implementation	49
16.4 Analysis: Approximations and Probability	50
16.5 Applications	51
16.6 Tight Example: Clause Size Bottleneck	51
16.7 Exercises	51
17 Chernoff Bounds	52
17.1 Intuition: Concentration and Coverage	52
17.2 Problem Definition	52
17.3 Algorithm and Python Implementation	52
17.4 Analysis: Chernoff Bounds and Set Cover	53
17.5 Applications	53
17.6 Tight Example: The Grid Partition	54
17.7 Exercises	54
18 Semidefinite Programming	55
18.1 Intuition: High-Dimensional Embeddings and Hyperplanes	55
18.2 Problem Definition	55
18.3 Algorithm and Python Implementation	55
18.4 Analysis: The 0.878 Performance Bound	56
18.5 Applications	57
18.6 Tight Example: The 5-Cycle Graph	57
18.7 Exercises	57

19 Multiway Cut LP Rounding	58
19.1 Intuition: Simplex-Distance LP and CKR Rounding	58
19.2 Problem Definition	58
19.3 Algorithm and Python Implementation	58
19.4 Analysis: Simplex Embedding and 1.5 Bound	59
19.5 Applications	60
19.6 Tight Example: The Spokes Graph	60
19.7 Exercises	60
21 Steiner Forest	61
21.1 Intuition: Connective Components and Reverse Deletion	61
21.2 Problem Definition	61
21.3 Algorithm and Python Implementation	61
21.4 Analysis: Factor 2 Approximation	62
21.5 Applications	63
21.6 Tight Example: The Redundant Cycle	63
21.7 Exercises	64
26 Max 2-SAT	65
26.1 Intuition: Anchoring with a True Variable	65
26.2 Problem Definition	65
26.3 Algorithm and Python Implementation	65
26.4 Analysis: The SDP Relaxation and LLZ Rounding	66
26.5 Applications	67
26.6 Tight Example: The All-Combinations Cycle	67
26.7 Exercises	67
22 Steiner Network	68
22.1 Intuition: LP Extreme Points and Iterative Rounding	68
22.2 Problem Definition	68
22.3 Algorithm and Python Implementation	68
22.4 Analysis: Jain's Theorem and Factor 2	69
22.5 Applications	70
22.6 Tight Example: The Multi-Path Cycle	70
22.7 Exercises	70
23 Feedback Vertex Set via Primal-Dual	71
23.1 Intuition: Cycle Hitting and Vertex Budgets	71
23.2 Problem Definition	71
23.3 Algorithm and Python Implementation	71
23.4 Analysis: Dual Pressure and Factor 2	73
23.5 Applications	73
23.6 Tight Example: Intersecting Loops	74
23.7 Exercises	74
24 Facility Location	75
24.1 Intuition: Primal-Dual with Filters	75
24.2 Problem Definition	75

24.3 Primal-Dual 3-Approximation	75
24.3.1 Other Algorithms	76
24.4 Python Implementation	76
24.5 Applications	77
24.6 Exercises	77
30 Multicut in Trees	78
30.1 Intuition: LCA Depth Sorting and Dual Growth	78
30.2 Problem Definition	78
30.3 Algorithm and Python Implementation	78
30.4 Analysis: Factor 2 Approximation	79
30.5 Applications	80
30.6 Tight Example: The Branching LCA	80
30.7 Exercises	80

Chapter 1

Introduction

1.1 The Approximation Paradigm

Intuition 1.1. The Core Problem: Most natural optimization problems are NP-hard. Assuming $P \neq NP$, we cannot find exact solutions efficiently. But we can find near-optimal solutions with provable guarantees.

The Key Insight: To prove a guarantee like “within $2\times$ optimal,” we don’t need to know the optimal value. We just need a lower bound on OPT that we can compute efficiently. If our algorithm’s solution is within a factor of that lower bound, we have our guarantee.

Definition 1.1 (α -Approximation Algorithm). For a minimization problem, an algorithm A is an α -approximation ($\alpha \geq 1$) if for every instance I :

$$A(I) \leq \alpha \cdot \text{OPT}(I)$$

where $\text{OPT}(I)$ is the optimal value. For maximization: $A(I) \geq \frac{1}{\alpha} \text{OPT}(I)$.

1.1.1 Vertex Cover: A Gentle Beginning

Definition 1.2 (Vertex Cover). Given an undirected graph $G = (V, E)$, a **vertex cover** is a subset $C \subseteq V$ such that every edge has at least one endpoint in C . The **cardinality vertex cover problem** asks for a cover of minimum size.

1.1.2 Lower Bound via Maximal Matching

Definition 1.3 (Matching). A **matching** $M \subseteq E$ is a set of edges with no shared endpoints. It is **maximal** if no edge can be added without violating the matching property.

Lemma 1.4. For any graph G , the size of a maximal matching M is a lower bound on the minimum vertex cover: $|M| \leq \text{OPT}$.

Proof. Each edge in M needs a distinct vertex in any vertex cover (edges don’t share endpoints). So $|C| \geq |M|$ for any cover C . $\square$

Intuition 1.2. *Intuition:* A maximal matching is easy to find greedily. It pairs up vertices that “must be covered” in some sense. Each pair contributes 2 vertices to our cover but at most 1 to the optimal cover. This 2:1 ratio is the source of the factor-2 guarantee.

1.1.3 The 2-Approximation Algorithm

Algorithm 1.5 (Vertex Cover via Maximal Matching (Factor 2)). 1. Find a maximal matching M in G (greedily).

2. Output $C = \{u, v \mid (u, v) \in M\}$ — all matched vertices.

Theorem 1.6. Algorithm 1.1 is a factor-2 approximation for Vertex Cover.

Proof. C is a vertex cover because M is maximal (any uncovered edge could be added to M). Since $|M| \leq \text{OPT}$ and $|C| = 2|M|$, we have $|C| \leq 2\text{OPT}$. $\square$

1.1.4 Why Approximation? Why Not Exact?

- **Exact Vertex Cover is NP-complete** (Karp, 1972). No polynomial-time exact algorithm exists unless $P = NP$.
- **Fixed-parameter algorithms** exist ($O(1.2738^k + kn)$) but exponential in k .
- **Integer Linear Programming** solves it exactly but worst-case exponential.
- **Approximation gives polynomial time with a guarantee** — often the best practical choice.

1.1.5 Other Approaches and Their Limits

Table 1.1: Approaches to Vertex Cover

Method	Time	Guarantee
Brute force / ILP	Exponential	Exact
Fixed-parameter	$O(1.2738^k)$	Exact, but exponential in k
Maximal matching (this chapter)	$O(m)$	Factor 2
LP rounding (Chapter 14)	Poly	Factor 2
Semidefinite programming	Poly	Factor $2 - \epsilon$ (open)

Why not better than 2? Under the Unique Games Conjecture, no polynomial-time algorithm can achieve $(2 - \epsilon)$ -approximation (Khot-Regev, 2008). The factor 2 is essentially tight.

1.1.6 Python Implementation

```

1 def maximal_matching(graph):
2     """Greedy maximal matching: pick edges, remove endpoints."""
3     n = len(graph)
4     matched = [False] * n

```

```

5  matching = []
6  for u in range(n):
7      if not matched[u]:
8          for v in graph[u]:
9              if not matched[v]:
10                 matching.append((u, v))
11                 matched[u] = matched[v] = True
12                 break
13  return matching
14
15 def vertex_cover_approx_2(graph):
16     """Factor-2 approximation for Vertex Cover."""
17     matching = maximal_matching(graph)
18     cover = set()
19     for u, v in matching:
20         cover.add(u)
21         cover.add(v)
22     return cover

```

Listing 1.1: Vertex Cover 2-Approximation

1.2 Applications

- **Network monitoring:** Place sensors on vertices to monitor all links
- **Compiler register allocation:** Variables as vertices, conflicts as edges
- **Bioinformatics:** Protein interaction networks — find minimal interacting set
- **Scheduling:** Tasks as vertices, conflicts as edges

Real Example: Register Allocation in a Compiler

Scenario: A compiler must assign $n = 50$ variables to $k = 16$ CPU registers. Variables used simultaneously cannot share a register. **Model:** Conflict graph $G = (V, E)$ where $V =$ variables, $(u, v) \in E$ if u, v are live simultaneously. Minimum vertex cover finds the minimum set of variables to *spill to memory* so the remaining k can fit in registers. **Why 2-approx matters:** If OPT spills 10 variables, our algorithm spills ≤ 20 . In practice, maximal matching often finds near-optimal solutions. The guarantee means we never spill more than twice the minimum — critical for register-starved architectures (embedded systems). **Alternative approach:** ILP (exact) takes exponential time; greedy degree-based gives $O(\log n)$.

1.2.1 Tight Example

Example 1.7 (Complete Bipartite $K_{n,n}$). *The algorithm picks all $2n$ vertices. Optimal picks one side (n vertices). Ratio = 2.*

1.2.2 Exercises

Exercise 1.8. *Show that the greedy algorithm picking highest-degree vertices gives $O(\log n)$ approximation.*

Exercise 1.9. *Design a 2-approximation for minimum cardinality maximal matching.*

Chapter 2

Set Cover

2.1 Intuition: The Greedy Instinct

Intuition 2.1. The Greedy Idea: When you need to cover all elements, pick the set that gives you the most “bang for your buck” — the most new elements per unit cost. Repeat until everything is covered.

Why it works: The price paid for each element when it’s covered serves as a dual variable. Summing all prices gives a lower bound on OPT (via LP duality, Chapter 13). The harmonic number H_n emerges naturally because early elements get covered cheaply, later elements pay more.

2.2 Problem Definition

Definition 2.1 (Set Cover). Given a universe $U = \{e_1, \dots, e_n\}$ and a collection of subsets $\mathcal{S} = \{S_1, \dots, S_m\}$ with costs $c(S_i) \geq 0$, find a minimum-cost subcollection $\mathcal{C} \subseteq \mathcal{S}$ such that $\bigcup_{S \in \mathcal{C}} S = U$.

2.3 The Greedy Algorithm

Algorithm 2.2 (Greedy Set Cover — H_n Approximation). 1. $C \leftarrow \emptyset$ (covered elements)

2. While $C \neq U$:

(a) Pick $S \in \mathcal{S}$ minimizing $\frac{c(S)}{|S \setminus C|}$ (cost-effectiveness)

(b) For each $e \in S \setminus C$, set $\text{price}(e) = \frac{c(S)}{|S \setminus C|}$

(c) $C \leftarrow C \cup S$

3. Output the picked sets

Theorem 2.3. The Greedy Set Cover algorithm achieves approximation factor $H_n = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \ln n + \gamma$.

Proof Sketch. Each element’s price is a lower bound on its contribution to OPT . The k -th element covered pays at most $\frac{OPT}{k}$. Summing gives $H_n \cdot OPT$. $\square$

2.3.1 Layering: Another 2-Approximation for Weighted Vertex Cover

Layering decomposes vertex weights into degree-weighted functions. Each layer gives a factor-2 vertex cover, and summing gives factor-2 for the weighted case. This is a preview of the primal-dual schema (Chapter 22).

2.3.2 Application to Shortest Superstring

The Shortest Superstring problem (find shortest string containing all given strings as substrings) reduces to Set Cover via string overlaps. Greedy gives $2H_n$ -approximation; Li (1992) improved to factor 3.

2.4 Other Algorithms for Set Cover

Table 2.1: Approaches to Set Cover

Algorithm	Approximation	Technique
Greedy (this chapter)	H_n	Combinatorial
LP Rounding (Ch. 14)	f (max frequency)	LP + Rounding
Primal-Dual (Ch. 13, 22)	f	Primal-Dual Schema
Randomized Rounding	$O(\log n)$	LP + Randomization

Why Greedy? It's simple, fast, and achieves the optimal $\ln n$ bound (Feige, 1998 proved no $(1 - \epsilon) \ln n$ unless $P = NP$).

Why not better? Feige (1998) showed no polynomial-time algorithm can achieve $(1 - \epsilon) \ln n$ approximation unless $NP \subseteq DTIME(n^{\log \log n})$.

2.5 Python Implementation

```

1 def greedy_set_cover(universe, sets, costs):
2     """Hn-approximation for Set Cover."""
3     covered = set()
4     picked = []
5     total_cost = 0.0
6     prices = {}
7
8     while covered != universe:
9         best_set = None
10        best_ratio = float('inf')
11        for s_id, s in sets.items():
12            new = s - covered
13            if not new: continue
14            ratio = costs[s_id] / len(new)
15            if ratio < best_ratio:
16                best_ratio = ratio
17                best_set = s_id
18
19        if best_set is None: break

```

```

20     new_elements = sets[best_set] - covered
21     for e in new_elements:
22         prices[e] = best_ratio
23     covered |= sets[best_set]
24     picked.append(best_set)
25     total_cost += costs[best_set]
26
27     return picked, total_cost

```

Listing 2.1: Greedy Set Cover

2.6 Applications

- **Feature selection:** Minimum features covering all classes
- **Antenna placement:** Cover all locations with minimum transmitters
- **Test case minimization:** Cover all code paths with fewest tests
- **Emergency facilities:** Fire stations covering all districts
- **Data compression:** Shortest superstring via set cover

Real Example: A/B Test Minimization at a Tech Company

Scenario: A/B testing platform runs 10,000 experiments. Each experiment tests a variant against control on a subset of 100 metrics (click-through, revenue, retention, etc.). Running an experiment on all metrics is expensive. **Model:** Universe = 100 metrics. Sets = experiments (each covers ~ 15 metrics). Cost = compute cost per experiment. **Why greedy:** The greedy algorithm picks the experiment covering the most *new* metrics per unit cost. $H_{100} \approx 5.18$ means greedy uses $\leq 5 \times$ the minimum experiments. **Real result:** In practice, greedy reduces experiments by 70% vs. running all, with H_n bound giving stakeholders confidence. **Set cover variant:** *Budgeted maximum coverage* — given k experiments, maximize metrics covered. Greedy gives $(1 - 1/e)$ -approx (Chapter 13).

Real Example: Feature Selection for Medical Diagnosis

Scenario: A hospital has $n = 10,000$ patient features (lab results, vitals, history). They want to select the minimum subset that still predicts $m = 50$ diseases with 95% accuracy. **Model:** Universe U = diseases. Each feature f covers the subset $S_f \subseteq U$ it helps predict above threshold. Cost $c(f) = 1$ (each feature has equal acquisition cost). **Why greedy works here:** At each step, greedy picks the feature that adds the most *new* diseases predicted. The $H_{50} \approx 4.5$ guarantee means if optimal uses 20 features, greedy uses ≤ 90 . In practice, greedy often finds near-optimal because features have diminishing returns (submodularity). **Real constraint:** Some features are expensive (MRI) vs. cheap (blood pressure). Weighted set cover handles this — greedy picks by cost-effectiveness. **Alternative:** LASSO (convex relaxation) gives exact recovery under restricted isometry but no worst-case guarantee; ILP is exact but slow for $n = 10,000$.

2.7 Tight Example

Example 2.4. Universe $\{e_1, \dots, e_n\}$. Sets: $S_0 = U$ (cost $1 + \epsilon$), $S_i = \{e_i\}$ (cost $1/i$). Greedy picks all S_i (total H_n). Optimal picks S_0 (cost $1 + \epsilon$). Ratio $\rightarrow H_n$.

2.8 Exercises

Exercise 2.5. Show greedy achieves H_k where $k = \max_i |S_i|$.

Exercise 2.6. Give a factor- f algorithm where f is maximum element frequency.

Chapter 3

Steiner Tree and TSP

3.1 Intuition: Metric Closure and MST

Intuition 3.1. Steiner Tree Intuition: We only need to connect the terminals (required vertices). The shortest path between two terminals in the original graph defines a distance. Build the complete graph on terminals with these distances (the metric closure). The MST of this closure, when expanded back to shortest paths in the original graph, gives a tree connecting all terminals within factor 2.

TSP Intuition: The MST is a lower bound on TSP (removing any edge from a TSP tour gives a spanning tree). Double the MST to make it Eulerian, find an Euler tour, then shortcut repeated vertices. Triangle inequality ensures shortcuts don't increase cost. Christofides' insight: instead of doubling all edges, just add a matching on odd-degree vertices.

3.2 Metric Steiner Tree

Definition 3.1 (Metric Steiner Tree). Given graph $G = (V, E)$ with metric edge costs and terminals $R \subseteq V$, find minimum-cost tree connecting all terminals (may use Steiner vertices $V \setminus R$).

Algorithm 3.2 (Metric Steiner Tree — Factor 2). 1. Compute metric closure G' on terminals: complete graph with $d(u, v) = \text{shortest path distance in } G$

2. Find MST T' of G'
3. Replace each edge of T' with corresponding shortest path in G
4. Let H be the union; find MST T of H
5. Prune non-terminal leaves from T

Theorem 3.3. This algorithm achieves a factor-2 approximation for Metric Steiner Tree.

3.3 Metric TSP

Definition 3.4 (Metric TSP). Given complete graph with metric edge costs, find minimum-cost Hamiltonian cycle.

Without metric assumption, TSP has no polynomial-time approximation unless $P = NP$ (reduction from Hamiltonian Cycle).

3.3.1 Factor-2 Algorithm (Double Tree)

Algorithm 3.5 (Metric TSP — Factor 2). 1. Find MST T of G

2. Double every edge of T to get Eulerian multigraph

3. Find Eulerian tour

4. Shortcut: visit vertices in order of first appearance

Theorem 3.6. Double-tree gives factor-2 approximation for Metric TSP.

3.3.2 Christofides' 3/2-Approximation

Algorithm 3.7 (Christofides — Factor 3/2). 1. Find MST T

2. Let O be odd-degree vertices in T ($|O|$ is even)

3. Find minimum-weight perfect matching M on O (using metric closure distances)

4. $T \cup M$ is Eulerian; find Euler tour and shortcut

Theorem 3.8 (Christofides, 1976). Christofides' algorithm achieves a 3/2-approximation for Metric TSP.

Proof. $\text{Cost}(T) \leq \text{OPT}$ (MST lower bound). $\text{Cost}(M) \leq \text{OPT}/2$ (matching on odd vertices is at most half the optimal tour). Total $\leq \text{OPT} + \text{OPT}/2 = 1.5 \text{ OPT}$. $\square$

3.3.3 Other Approaches for TSP

Table 3.1: TSP Algorithms for Metric Case

Algorithm	Approximation	Technique
Double-tree	2	MST + Euler tour
Christofides	1.5	MST + Matching
Held-Karp (LP)	≤ 1.5	LP relaxation (exact value ≈ 1.09)
Iterative rounding	< 1.5	Recent breakthroughs (Karlin et al., 2021: $1.5 - \epsilon$)

Why not exact? Metric TSP is NP-hard (reduction from Hamiltonian Cycle). The best exact algorithm is $O(n^{2^{2n}})$ (Held-Karp DP).

Why 3/2 was a barrier? For 45 years no improvement. Karlin-Klein-Gharan (2021) broke it with $1.5 - 10^{-36}$, later improved to $1.5 - \epsilon$.

3.4 Python Implementation

```

1 def tsp_christofides(graph):
2     """3/2-approx for Metric TSP."""
3     n = len(graph)
4     if n <= 2: return tsp_2approx(graph)
5
6     # 1. MST
7     mst_edges = mst_prim(graph)
8
9     # 2. Odd-degree vertices
10    degree = {u: 0 for u in graph}
11    for u, v in mst_edges:
12        degree[u] += 1; degree[v] += 1
13    odd = [u for u, d in degree.items() if d % 2 == 1]
14
15    # 3. Min-weight perfect matching on odd vertices
16    closure = metric_closure(graph, set(odd))
17    matching = min_weight_perfect_matching(closure, odd)
18
19    # 4. Euler tour + shortcut
20    euler_adj = build_eulerian(mst_edges, matching)
21    tour = eulerian_tour(euler_adj)
22    ham = shortcut(tour)
23
24    return ham, cost(ham)

```

Listing 3.1: Christofides TSP 3/2-Approximation

3.5 Applications

- **VLSI design:** Steiner tree = wire routing; TSP = drill path optimization
- **Logistics:** Vehicle routing (TSP variants)
- **Phylogeny:** Steiner tree = evolutionary trees
- **Network design:** Connect required nodes at minimum cost
- **DNA sequencing:** Shortest superstring = genome assembly

Real Example: Vehicle Routing for E-Commerce Delivery

Scenario: A delivery company has 1 depot, 50 packages, 5 trucks. Each truck starts/ends at depot. Minimize total distance. **Model:** Complete graph on 51 nodes (depot + 50 customers). Edge costs = travel time (satisfies triangle inequality). Optimal TSP tour = optimal single-truck route. For 5 trucks: split TSP tour into 5 segments (each $\leq$ capacity). Metric TSP 1.5-approx (Christofides) gives route within $1.5\times$ optimal single-truck distance. **Why approximation matters:** Exact TSP (Held-Karp DP) is $O(n^2 2^n)$ — impossible for $n = 50$. Christofides runs in $O(n^3)$ and gives provable $1.5\times$ bound. In practice, Christofides + 2-opt local search often achieves $< 1.1\times$ optimal. **Steiner Tree variant:** If only 20 of 50 locations require delivery today, Steiner Tree connects the 20 "terminals" at minimum cost — useful for ad-hoc routing.

3.6 Tight Examples

Example 3.9 (Steiner Tree). *Star graph: center connected to n leaves (cost 1), leaf-to-leaf cost 2. Optimal = n (star). Algorithm gives $2n - 2$. Ratio $\rightarrow 2$.*

Example 3.10 (TSP Double-Tree). *Star with center 0, leaves $1..n$. Edges $(0,i)$ cost 1, (i,j) cost 2. MST = star $(n - 1)$. Double tree = $2n - 2$. Optimal tour = n ($0-1-2-\dots-n-0$). Ratio $\rightarrow 2$.*

Example 3.11 (Christofides). *Odd cycle of length n (odd). MST = path $(n - 1)$. Odd vertices = 2 ends. Matching adds $(n + 1)/2$. Total = $1.5n - 0.5$. Optimal = n . Ratio $\rightarrow 1.5$.*

3.7 Exercises

Exercise 3.12. *Show that in Euclidean plane, optimal Steiner tree has all Steiner points of degree 3 with 120° angles.*

Exercise 3.13. *Give a factor-2 algorithm for Rectilinear Steiner Arborescence.*

Chapter 4

Multiway Cut and k-Cut

4.1 Intuition: Isolating Cuts

Intuition 4.1. Multiway Cut: We have k terminals that must be separated. For each terminal s_i , find the minimum cut isolating s_i from all other terminals (merge other terminals into a super-terminal). Take the union of the $k - 1$ lightest isolating cuts.

Why it works: In the optimal multiway cut, each terminal sits in its own component. The cut separating component i from the rest is an isolating cut for s_i . The sum of all k isolating cuts in OPT is exactly $2 \times OPT$ (each edge counted twice). Discarding the heaviest leaves at most $2(1 - 1/k)OPT$.

4.2 Multiway Cut

Definition 4.1 (Multiway Cut). Given graph $G = (V, E)$ with edge weights and terminals $S = \{s_1, \dots, s_k\}$, find minimum-weight edge set whose removal disconnects all terminals from each other.

Algorithm 4.2 (Multiway Cut — Factor $2 - 2/k$). 1. For each $i = 1..k$, compute minimum isolating cut C_i separating s_i from $S \setminus \{s_i\}$

2. Discard the heaviest C_i

3. Output union of remaining $k - 1$ cuts

Theorem 4.3. This algorithm achieves a $2 - \frac{2}{k}$ approximation for Multiway Cut.

4.2.1 Minimum k-Cut

Definition 4.4 (k -Cut). Find minimum-weight edge set whose removal partitions G into at least k components.

The Gomory-Hu tree represents all pairwise minimum cuts in a tree with $n - 1$ edges. Removing the $k - 1$ lightest edges gives a k -cut.

Algorithm 4.5 (Minimum k -Cut — Factor $2 - 2/k$). 1. Compute Gomory-Hu tree T of G

2. Remove $k - 1$ lightest edges from T

3. Output union of corresponding cuts in G

Theorem 4.6. *This achieves a $2 - \frac{2}{k}$ approximation for Minimum k -Cut.*

4.2.2 Other Approaches

Table 4.1: Multiway Cut / k -Cut Algorithms

Algorithm	Approximation	Technique
Isolating cuts (this chapter)	$2 - 2/k$	Min-cut subroutine
Gomory-Hu tree (this chapter)	$2 - 2/k$	Min-cut + tree
LP rounding (Ch. 19)	1.5	LP relaxation
Region growing	2	Continuous relaxation

4.3 Python Implementation

```

1 def multiway_cut_2_2k(graph, terminals):
2     """2-2/k approx for Multiway Cut."""
3     cuts = []
4     weights = []
5     for s in terminals:
6         other = terminals - {s}
7         S, T = min_isolating_cut(graph, s, other)
8         cuts.append((S, T))
9         weights.append(cut_weight(graph, S, T))
10
11     max_idx = max(range(len(terminals)), key=lambda i: weights[i])
12     result = set()
13     for i, (S, T) in enumerate(cuts):
14         if i != max_idx:
15             result.update(cut_edges(graph, S, T))
16     return result
17
18 def min_k_cut_2_2k(graph, k):
19     """2-2/k approx for Minimum k-Cut."""
20     tree = gomory_hu_tree(graph)
21     lightest = sorted(tree.edges, key=lambda e: e.weight)[:k-1]
22     return union_of_cuts(graph, lightest)

```

Listing 4.1: Multiway Cut and k -Cut

4.4 Applications

- **Image segmentation:** Multiway cut = partition pixels into regions
- **Network reliability:** k -cut = minimum edges to disconnect into k parts
- **Clustering:** Multiway cut with similarity weights
- **VLSI partitioning:** Divide circuit into modules

Real Example: Medical Image Segmentation

Scenario: MRI brain scan with $k = 4$ tissue types (gray matter, white matter, CSF, tumor). Each pixel must be assigned to one type. **Model:** Graph where vertices = pixels, edges = adjacency. Terminals = seed pixels labeled by radiologist for each tissue type. Edge weights = similarity (inverse intensity difference). Multiway cut separates the 4 terminals with minimum boundary cost = maximum coherence within each tissue region. **Why multiway cut:** Unlike k -means, multiway cut respects boundary continuity — cuts follow intensity gradients. The $2 - 2/4 = 1.5$ -approximation guarantees the segmentation boundary is at most $1.5\times$ the optimal length. **In practice:** With pre-seeded terminals from a CNN, the multiway cut refines boundaries better than pure CNN output (post-processing step in medical imaging pipelines). **k -cut variant:** No pre-labeled seeds — find k components of minimum boundary. Used for unsupervised segmentation.

4.5 Tight Example

$2k$ vertices in a cycle: k terminals alternating with k Steiner vertices. Terminal-Steiner edges weight $2 - \epsilon$, Steiner-Steiner edges weight 1. Algorithm picks $k - 1$ heavy edges; optimal picks k light edges. Ratio $\rightarrow 2 - 2/k$.

4.6 Exercises

Exercise 4.7. Show Algorithm 4.3 can find a k -cut within $2 - 2/k$ using multiway cut as subroutine.

Exercise 4.8. Prove the greedy multiway cut algorithm (iteratively remove lightest s_i - s_j cut) also achieves $2 - 2/k$.

Chapter 5

k-Center

5.1 Intuition: Parametric Pruning

Intuition 5.1. The Idea: We don't know the optimal radius r^* , but we can guess it. For a guess r , build graph G_r with edges of length $\leq r$. Can we cover all vertices with k centers of radius r ? This means finding a dominating set of size k in G_r . Checking this is NP-hard, but we can approximate: find a maximal independent set in G_r^2 (square graph). If its size $\leq k$, its neighborhoods in G_r cover everything.

Why Square Graph? If M is an MIS in G^2 , then every vertex is within distance 2 of M in G . In metric spaces, distance 2 means radius $\leq 2r$ in the original metric. So we get a 2-approximation.

5.2 Problem Definition

Definition 5.1 (Metric k -Center). Given complete graph with metric distances, choose k centers minimizing the maximum distance from any vertex to its nearest center:

$$\min_{S \subseteq V, |S|=k} \max_{v \in V} \min_{u \in S} \text{dist}(u, v)$$

This is the *warehouse location* problem: minimize worst-case delivery distance.

5.2.1 Parametric Pruning Algorithm

Algorithm 5.2 (Metric k -Center — Factor 2). 1. Sort distinct edge weights; for each threshold r , build G_r (edges $\leq r$)

2. For each G_r , compute G_r^2 (square: edges for distance ≤ 2)

3. Find maximal independent set M in G_r^2

4. If $|M| \leq k$ and M 's neighborhoods in G_r cover all vertices, output neighborhoods as centers with radius $2r$

Theorem 5.3. This algorithm achieves a factor-2 approximation for Metric k -Center.

5.2.2 Weighted k -Center

Vertices have weights; total weight of centers $\leq W$. Same parametric pruning with weight-aware selection gives factor-3 approximation.

5.2.3 Other Approaches

Table 5.1: k -Center Algorithms

Algorithm	Approximation	Technique
Parametric pruning (this chapter)	2	Combinatorial
Farthest-first traversal	2	Greedy
LP rounding	2	LP relaxation
High-dimensional ℓ_p	$1 + \epsilon$	Coresets

Why not exact? k -Center is NP-hard even for $k = 2$ (reduction from Dominating Set). Without metric assumption, no $\alpha(n)$ -approximation exists.

5.3 Python Implementation

```

1 def kcenter_parametric_pruning(graph, k):
2     """2-approx for k-Center via parametric pruning."""
3     thresholds, graphs = build_graphs_by_threshold(graph)
4
5     for t, g in zip(thresholds, graphs):
6         g2 = graph_square(g)
7         mis = maximal_independent_set(g2)
8
9         if len(mis) <= k:
10             centers = set(mis)
11             # Pad if we have fewer than k centers
12             if len(centers) < k:
13                 remaining = set(graph.keys()) - centers
14                 centers.update(list(remaining)[:k - len(centers)])
15             return centers, t
16
17     return set(graph.keys()), max(thresholds)

```

Listing 5.1: k -Center Parametric Pruning

5.4 Applications

- **Facility location:** Fire stations, warehouses, cell towers
- **Clustering:** Minimize cluster radius (k -center = bottleneck k -means)
- **Load balancing:** Distribute tasks to minimize max load
- **Robotics:** Multi-robot coverage of an area

Real Example: Cell Tower Placement for 5G

Scenario: Telecom placing $k = 15$ new 5G towers in a city of $n = 2,000$ potential sites. Each site covers a radius r . Goal: minimize r so every subscriber is covered. **Model:** Complete graph with metric distances (lat/long $\rightarrow$ haversine). k -center finds k centers minimizing maximum distance to nearest center = maximum subscriber-tower distance. **Why 2-approx matters:** If OPT radius = 800m, our towers have radius ≤ 1.6 km. Guarantee ensures no subscriber is more than $2\times$ the minimum possible distance from a tower. **Real constraint:** Some sites are unavailable (zoning, cost). Remove those vertices from graph — parametric pruning handles this naturally. **Comparison:** k -means minimizes *sum* of squared distances (average case), k -center minimizes *maximum* distance (worst case). For coverage guarantees, k -center is correct model. **Weighted k -center (Ch. 5):** Towers have different capacities (weights). Total capacity budget W . 3-approx algorithm extends directly.

5.5 Tight Example: Wheel Graph

Center connected to n outer vertices (weight 1), outer cycle weight 2. For $k = 1$, optimal center = hub (radius 1). Algorithm may pick outer vertex (radius 2). Ratio = 2.

5.6 Exercises

Exercise 5.4. Show that without triangle inequality, k -center has no $\alpha(n)$ -approximation.

Exercise 5.5. Modify Algorithm 5.3 to use minimal dominating set instead of MIS in G_i^2 .

Chapter 6

Feedback Vertex Set

6.1 Intuition: Peeling and Layering

Intuition 6.1. Peeling and Layering Intuition: Feedback Vertex Set (FVS) is the problem of breaking all cycles in a graph by deleting a minimum-weight set of vertices. Low-degree vertices cannot support cycles: degree 0 or 1 vertices are useless for cycles, and degree 2 vertices can be contracted. In a “clean” graph (minimum degree ≥ 3), the average degree of any forest is less than 2, meaning any cycle-free subgraph must connect heavily to the deleted feedback vertices. By reducing weights proportionally to a vertex’s cycle-breaking contribution (measured by its cyclomatic weight $\delta_G(v)$), we pay at most twice the optimal cost to hit every cycle.

6.2 Problem Definition

Given an undirected graph $G = (V, E)$ with non-negative vertex weights $w : V \rightarrow \mathbb{R}^+$, the *Minimum Feedback Vertex Set* (FVS) problem is to find a subset $F \subseteq V$ of minimum total weight $\sum_{v \in F} w(v)$ such that the induced subgraph $G[V \setminus F]$ contains no cycles (i.e., it is a forest).

The decision version of FVS is one of Karp’s 21 NP-complete problems. It has applications in database deadlock resolution, circuit design, and Bayesian network inference (loop cutsets).

6.3 Algorithm and Python Implementation

The 2-approximation algorithm uses the local-ratio technique. It first simplifies the graph by removing vertices of degree ≤ 1 and contracting vertices of degree 2. It then scales weights based on the cyclomatic weight $\delta_H(v) = \text{cyc}(H) - \text{cyc}(H \setminus \{v\})$, where $\text{cyc}(H) = |E(H)| - |V(H)| + \text{cc}(H)$ is the cyclomatic number of graph H .

```
1 def feedback_vertex_set_approx(graph, weights):
2     """2-approx for undirected weighted Feedback Vertex Set."""
3     g = {v: list(neighbors) for v, neighbors in graph.items()}
4
5     # 0. If any vertex has weight <= 0, remove and recurse
6     for v in list(g.keys()):
7         if weights.get(v, 0.0) <= 1e-9:
```

```

8         g_sub = {x: [y for y in neighbors if y != v] for x, neighbors
9                    in g.items() if x != v}
10        sub_fvs = feedback_vertex_set_approx(g_sub, weights)
11        sub_fvs.add(v)
12        # Prune before returning
13        final_fvs = set(sub_fvs)
14        for x in list(final_fvs):
15            if is_feedback_vertex_set(graph, final_fvs - {x}):
16                final_fvs.remove(x)
17        return final_fvs
18
19        # 1. Clean the graph (remove degree <= 1, contract degree 2)
20        changed = True
21        while changed:
22            changed = False
23            to_remove = [v for v in g if len(g[v]) <= 1]
24            if to_remove:
25                for v in to_remove:
26                    for nxt in g[v]:
27                        if nxt in g:
28                            while v in g[nxt]: g[nxt].remove(v)
29                    del g[v]
30            changed = True
31            continue
32
33        deg2_vertices = [v for v in g if len(g[v]) == 2]
34        for v in deg2_vertices:
35            if v not in g: continue
36            u, w = g[v]
37            if u != w:
38                del g[v]
39                g[u] = [w if x == v else x for x in g[u]]
40                g[w] = [u if x == v else x for x in g[w]]
41                changed = True
42                break
43
44        if not g or cyclomatic_number(g) == 0:
45            return set()
46
47        # 2. Check for self-loops (cycle of length 1)
48        for v in g:
49            if v in g[v]:
50                g_sub = {x: [y for y in neighbors if y != v] for x, neighbors
51                           in g.items() if x != v}
52                sub_fvs = feedback_vertex_set_approx(g_sub, weights)
53                sub_fvs.add(v)
54                return sub_fvs
55
56        # 3. Check for multi-edges (cycle of length 2)
57        for u in g:
58            for v in g[u]:
59                if g[u].count(v) > 1:
60                    eps = min(weights[u], weights[v])
61                    weights_next = dict(weights)

```

```

60         weights_next[u] -= eps
61         weights_next[v] -= eps
62         g_sub = {x: list(neighbors) for x, neighbors in g.items()}
63         sub_fvs = feedback_vertex_set_approx(g_sub, weights_next)
64         if u not in sub_fvs and v not in sub_fvs:
65             sub_fvs.add(u if weights_next[u] <= 1e-9 else v)
66         return sub_fvs
67
68     # 4. Normal local ratio step
69     cyc_g = cyclomatic_number(g)
70     deltas = {}
71     for v in g:
72         g_without_v = {x: [y for y in neighbors if y != v] for x,
73                          neighbors in g.items() if x != v}
74         deltas[v] = cyc_g - cyclomatic_number(g_without_v)
75         if deltas[v] <= 0: deltas[v] = 1
76
77     eps = min(weights[v] / deltas[v] for v in g)
78     weights_next = dict(weights)
79     for v in g:
80         weights_next[v] -= eps * deltas[v]
81
82     sub_fvs = feedback_vertex_set_approx(g, weights_next)
83     for v in [v for v in g if weights_next[v] <= 1e-9]:
84         if v not in sub_fvs: sub_fvs.add(v)
85
86     final_fvs = set(sub_fvs)
87     for v in list(final_fvs):
88         if is_feedback_vertex_set(graph, final_fvs - {v}):
89             final_fvs.remove(v)
90     return final_fvs

```

Listing 6.1: Local Ratio Algorithm for Feedback Vertex Set

6.4 Analysis: Local Ratio and Cyclomatic Weights

The cyclomatic number $\text{cyc}(G) = |E| - |V| + \text{cc}(G)$ represents the maximum number of independent cycles in G . The value $\delta_G(v) = \text{cyc}(G) - \text{cyc}(G \setminus \{v\})$ measures how many cycles are broken by deleting v . We have:

$$\delta_G(v) = d_G(v) - 1 - (k(G \setminus \{v\}) - k(G))$$

Since a minimal feedback vertex set F leaves a forest $T = G[V \setminus F]$, the average degree in T is less than 2. From this, we can show that the sum of cyclomatic weights of any minimal FVS is bounded:

$$\sum_{v \in F} \delta_G(v) \leq 2 \cdot \text{OPT}_{IP}$$

By the Local Ratio Theorem, decomposing the weights $w(v)$ into $w'(v) + \epsilon \cdot \delta_G(v)$ guarantees that the final minimal FVS satisfies $w(F) \leq 2 \cdot \text{OPT}$.

6.5 Applications

- **Bayesian Inference (Loop Cutsets):** Pearl's belief propagation algorithm is exact on trees. For graphs with cycles, belief propagation can be run by conditioning on a loop cutset (which is an FVS). Finding a minimum-weight FVS minimizes the conditioning overhead.
- **Deadlock Resolution:** In database transactions, the wait-for graph tracks transaction dependencies. Cycles represent deadlocks. An FVS represents a set of transactions to abort to resolve all deadlocks with minimal cost.
- **Dependency Resolution in Compilers:** Breaking cyclic package dependencies in package managers or modular architectures.

Real Example: Deadlock Resolution in Distributed Databases

Scenario: A database has $n = 100$ concurrent transactions. Cyclic dependencies (deadlocks) occur when transaction A waits for B , B waits for C , and C waits for A . Aborting a transaction has a weight (representing processing time lost). We want to break all deadlocks with minimal total weight aborted. **Model:** Directed graph of dependencies. For undirected cycles (simplifying directed FVS to undirected FVS when cycles are symmetric), the FVS algorithm finds the minimum-cost set of transactions to abort. **Why 2-approx matters:** Minimizing transaction abort cost is NP-hard. A 2-approximation guarantees that we abort transactions costing no more than $2\times$ the absolute minimum abort cost required to free the database from deadlocks.

6.6 Tight Example: Bipartite Graph

For a bipartite graph $K_{3,3}$ where all vertex weights are 1, the optimal FVS is of size 2. If the algorithm selects a suboptimal set or order, the local ratio step can yield a set of size 4 if vertices are not chosen carefully. The ratio approaches 2 for larger instances.

6.7 Exercises

Exercise 6.1. *Prove that for any tree T , the Feedback Vertex Set is empty.*

Exercise 6.2. *Prove that for any graph G , the average degree of a forest $T \subset G$ is less than 2.*

Chapter 7

Shortest Superstring

7.1 Intuition: Overlaps and Cycles

Intuition 7.1. *Overlap Minimization Intuition:* Finding the shortest common superstring of a set of strings is equivalent to maximizing the total overlap between adjacent strings. If we construct a directed graph where edge weights represent the cost of transitioning between strings (i.e. prefix length of the successor not overlapping with the predecessor), the shortest superstring is a TSP-like path. While TSP path cover is hard, a cycle cover of minimum weight can be found in polynomial time. By opening the cycles of the optimal cycle cover, we obtain a set of representative strings which can then be merged to yield a guaranteed 4-approximation (or 3-approximation using greedy merges).

7.2 Problem Definition

Given a set of strings $S = \{s_1, \dots, s_n\}$ over a finite alphabet Σ , the *Shortest Common Superstring* (SCS) problem is to find a string s of minimum length $|s|$ such that each $s_i \in S$ is a substring of s .

Without loss of generality, we assume that no string $s_i \in S$ is a substring of any other string $s_j \in S$. The SCS problem is NP-hard and has direct applications in bioinformatics (de novo genome assembly) and text compression.

7.3 Algorithm and Python Implementation

The approximation algorithms construct a directed overlap graph where vertices are the input strings. The edge weight $w(u, v) = |v| - \text{ov}(u, v)$ is the cost of appending v after u , where $\text{ov}(u, v)$ is the length of the longest suffix of u that is a prefix of v .

```
1 def shortest_superstring_3approx(strings):
2     """3-approx algorithm for Shortest Superstring (Modified Greedy)."""
3     cleaned = preprocess_substrings(strings)
4     if not cleaned: return ""
5     if len(cleaned) == 1: return cleaned[0]
6
7     n = len(cleaned)
8     cost_matrix = [[0.0] * n for _ in range(n)]
```

```

9     for i in range(n):
10         for j in range(n):
11             cost_matrix[i][j] = len(cleaned[j]) - compute_overlap(cleaned[
12                 i], cleaned[j])
13
14     perm, _ = find_minimum_cycle_cover(cost_matrix)
15     cycles = extract_cycles(perm)
16     cycle_strings = cycle_cover_to_strings(cycles, cleaned)
17     return greedy_superstring(cycle_strings)

```

Listing 7.1: Approximation Algorithms for Shortest Superstring

7.4 Analysis: Cycle Covers as Lower Bounds

Let OPT be the length of the shortest superstring. We define the prefix distance function $d(u, v) = |v| - \text{ov}(u, v)$. This distance function is metric-like: it satisfies the triangle inequality $d(u, w) \leq d(u, v) + d(v, w)$.

A cycle cover of the overlap graph is a set of vertex-disjoint cycles covering all vertices. The minimum cycle cover has total weight $\text{wt}(C) \leq \text{OPT}$. By breaking each cycle c_k at an arbitrary string to form a single merged string s'_{c_k} , we can show that the sum of lengths of the cycle representative strings is bounded by:

$$\sum_{c_k} |s'_{c_k}| \leq \text{OPT} + \text{wt}(C) \leq 2 \cdot \text{OPT}$$

The 4-approximation simply concatenates these representative strings (giving at most $2 \cdot 2 \cdot \text{OPT} = 4 \cdot \text{OPT}$). The 3-approximation (Modified Greedy) runs a greedy merge on these representatives, which limits the overlap loss and achieves a factor of 3.

7.5 Applications

- **DNA Sequencing (De Novo Assembly):** Sanger sequencing and high-throughput sequencing generate millions of short DNA fragments (reads). The genome is assembled by finding the shortest common superstring of these reads.
- **Data Compression:** Storing a dictionary of common words/patterns as a single superstring and referencing substrings via offsets.

Real Example: Sanger DNA Sequence Assembly

Scenario: A biologist sequences a gene and gets 5 short overlapping reads: CATG, ATGT, TGTA, GTAC, and TACA. Goal: reconstruct the original DNA sequence. **Model:** Overlap graph vertices are the 5 reads. Minimum cycle cover groups overlapping fragments. The 3-approximation algorithm merges cycle representatives greedily to yield the reconstructed sequence: CATGTACA. **Why 3-approx matters:** Finding the absolute shortest common superstring is NP-hard. The 3-approximation ensures that the reconstructed sequence length is within a factor of 3 of the optimal assembly, preventing over-compaction or redundant sequence generation.

7.6 Tight Example: Greedy Overlap Heuristic

A tight example for the standard greedy algorithm consists of strings $s_1 = a^k b$, $s_2 = ba^k$, and $s_3 = a^k$. The greedy heuristic can make choices that result in a superstring of length close to 2 times optimal. The modified greedy cycle-cover method resolves these structural bottlenecks.

7.7 Exercises

Exercise 7.1. *Prove that the distance function $d(u, v) = |v| - ov(u, v)$ satisfies the triangle inequality $d(u, w) \leq d(u, v) + d(v, w)$.*

Exercise 7.2. *Explain why the minimum cycle cover can be solved in polynomial time using bipartite matching.*

Chapter 8

Knapsack and FPTAS

8.1 Intuition: Scaling Values

Intuition 8.1. FPTAS Idea: Knapsack has a pseudo-polynomial DP in $O(nW)$ time. If we scale down the values by a factor K , the DP runs in $O(n^2/K)$ time. The optimal value decreases by at most nK . Setting $K = \epsilon V_{\max}/n$ gives $(1 - \epsilon)$ -approximation in $O(n^3/\epsilon)$ time.

8.2 Problem Definition

Definition 8.1 (0/1 Knapsack). Given n items with weights w_i and values v_i , capacity W , maximize $\sum v_i x_i$ s.t. $\sum w_i x_i \leq W$, $x_i \in \{0, 1\}$.

8.2.1 Exact DP (Pseudo-polynomial)

$dp[i][w]$ = max value using first i items with weight $\leq w$. Runs in $O(nW)$.

8.2.2 FPTAS via Value Scaling

Algorithm 8.2 (Knapsack FPTAS — $(1 - \epsilon)$ -Approximation). 1. Let $V_{\max} = \max_i v_i$, $K = \frac{\epsilon V_{\max}}{n}$

2. Scale values: $v'_i = \lfloor v_i/K \rfloor$

3. Run exact DP on scaled instance: $dp[v] = \min$ weight to achieve scaled value v

4. Find max v with $dp[v] \leq W$, reconstruct solution

Theorem 8.3. The solution value is at least $(1 - \epsilon)\text{OPT}$.

Proof. Each value reduced by $< K$. Total reduction $\leq nK = \epsilon V_{\max} \leq \epsilon \text{OPT}$. □

8.2.3 Other Approaches

Table 8.1: Knapsack Algorithms

Algorithm	Approximation	Time
Exact DP	Exact	$O(nW)$
FPTAS (value scaling)	$1 - \epsilon$	$O(n^3/\epsilon)$
FPTAS (weight scaling)	$1 - \epsilon$	$O(n^2/\epsilon)$
Greedy by value/weight	2 (fractional)	$O(n \log n)$

Why not exact? Knapsack is weakly NP-hard. Exact requires pseudo-polynomial time. FPTAS gives polynomial time for any fixed ϵ .

8.3 Python Implementation

```

1 def knapsack_fptas(weights, values, capacity, epsilon):
2     """(1-epsilon)-approx for 0/1 Knapsack."""
3     n = len(weights)
4     Vmax = max(values)
5     K = epsilon * Vmax / n
6     if K == 0: K = 1
7
8     scaled = [int(v / K) for v in values]
9     max_scaled = sum(scaled)
10
11     # dp[v] = min weight to achieve value v
12     dp = [float('inf')] * (max_scaled + 1)
13     dp[0] = 0
14     choice = [[False] * (max_scaled + 1) for _ in range(n + 1)]
15
16     for i in range(1, n + 1):
17         v, w = scaled[i-1], weights[i-1]
18         for val in range(max_scaled, v - 1, -1):
19             if dp[val - v] + w < dp[val]:
20                 dp[val] = dp[val - v] + w
21                 choice[i][val] = True
22
23     best_val = max(v for v in range(max_scaled + 1) if dp[v] <= capacity)
24
25     # Reconstruct
26     selected = []
27     val = best_val
28     for i in range(n, 0, -1):
29         if choice[i][val]:
30             selected.append(i-1)
31             val -= scaled[i-1]
32
33     return selected, sum(values[i] for i in selected)

```

Listing 8.1: Knapsack FPTAS

8.4 Applications

- **Resource allocation:** Budget-constrained project selection
- **Portfolio optimization:** Risk-constrained investment
- **Logistics:** Cargo loading with weight limit
- **Cloud computing:** VM placement with capacity constraints

Real Example: IT Budget Allocation for Cloud Migration

Scenario: A company has \$500,000 budget to migrate $n = 200$ on-premise applications to the cloud. Each app i has migration cost w_i and business value v_i (revenue impact, risk reduction). Maximize total value within budget. **Why FPTAS:** Exact DP with $W = 500,000$ takes $O(nW) = 10^8$ operations — too slow for interactive planning. FPTAS with $\epsilon = 0.05$ runs in $O(n^3/\epsilon) \approx 1.6 \times 10^6$ operations — instant. **Guarantee:** The selected apps achieve $\geq 95\%$ of optimal business value. CFO can trust the plan. **Real constraint:** Some apps are interdependent (if A moves, B must move). This is *knapsack with dependencies* — can be modeled as tree-knapsack with FPTAS. **Alternative:** Greedy by v_i/w_i ratio gives 2-approx for fractional but can be arbitrarily bad for 0/1 (e.g., one item with value 100, weight 101; budget 100; greedy picks nothing).

8.5 Exercises

Exercise 8.4. Implement the FPTAS using weight scaling instead of value scaling.

Exercise 8.5. Show that the greedy algorithm by value/weight ratio gives a 2-approximation for fractional knapsack but can be arbitrarily bad for 0/1 knapsack.

Chapter 9

Bin Packing

9.1 Intuition: Fitting and Grouping

Intuition 9.1. Peeling and Layering Intuition: The goal of Bin Packing is to pack a set of items of various sizes into the minimum number of unit-capacity bins. Online algorithms like Next-Fit (which keeps only one open bin) and First-Fit (which places an item in the first bin that can hold it) achieve constant factor approximations (2 and 1.7, respectively). To do better offline, we can construct an Asymptotic PTAS (APTAS) using linear grouping. By separating out small items ($< \epsilon$) and grouping the remaining large items into $1/\epsilon^2$ groups, we can round item sizes up to the maximum in their group. This reduces the problem to a constant number of distinct sizes, which can be solved exactly using dynamic programming.

9.2 Problem Definition

Given a list of n items with sizes $s_1, \dots, s_n \in (0, 1]$, the *Bin Packing* problem is to pack all items into the minimum number of bins, each of capacity 1.

The problem is NP-hard. It is also NP-hard to approximate within a factor of $3/2 - \epsilon$ for any $\epsilon > 0$ in the absolute sense (due to a partition reduction). However, it admits an *Asymptotic PTAS* (APTAS) which packs items into at most $(1 + \epsilon)\text{OPT} + O(1/\epsilon^2)$ bins.

9.3 Algorithm and Python Implementation

We implement standard online algorithms (Next-Fit, First-Fit) and the offline First-Fit Decreasing heuristic. We also implement the Asymptotic PTAS (APTAS) using linear grouping, which rounds large item sizes and solves the rounded instance exactly using dynamic programming.

```
1 def bin_packing_aptas(items, eps=0.3, capacity=1.0):
2     """APTAS for Bin Packing via linear grouping."""
3     large_items = [x for x in items if x >= eps]
4     small_items = [x for x in items if x < eps]
5
6     if not large_items:
7         return first_fit(small_items, capacity)
8
```

```

9   large_items.sort()
10  n_large = len(large_items)
11  k = int(1.0 / (eps ** 2))
12  q = n_large // k
13
14  rounded_large = []
15  if q == 0 or k == 0:
16      rounded_large = list(large_items)
17  else:
18      groups = [large_items[i*q : (i+1)*q if i < k-1 else n_large] for i
19                  in range(k)]
19      for group in groups:
20          max_size = max(group)
21          rounded_large.extend([max_size] * len(group))
22
23  distinct_sizes = sorted(list(set(rounded_large)))
24  counts = tuple(rounded_large.count(s) for s in distinct_sizes)
25  configs = generate_configurations(distinct_sizes, capacity)
26  large_bins = pack_large_dp(counts, configs, distinct_sizes)
27
28  # Map rounded items back to original large items
29  flattened = sorted([item for b in large_bins for item in b])
30  item_map = {s: [] for s in distinct_sizes}
31  for r_val, orig_val in zip(flattened, large_items):
32      item_map[r_val].append(orig_val)
33
34  final_large_bins = []
35  for b in large_bins:
36      new_bin = [item_map[item].pop(0) for item in b]
37      final_large_bins.append(new_bin)
38
39  # Place small items using First-Fit
40  for item in small_items:
41      placed = False
42      for b in final_large_bins:
43          if sum(b) + item <= capacity:
44              b.append(item)
45              placed = True
46              break
47      if not placed:
48          final_large_bins.append([item])
49  return final_large_bins

```

Listing 9.1: Asymptotic PTAS for Bin Packing

9.4 Analysis: Linear Grouping and APTAS

For Next-Fit, we can prove that the number of bins $A(I) \leq 2 \cdot \text{OPT}(I) + 1$, because the sum of items in any two consecutive bins must exceed 1.

The Asymptotic PTAS (APTAS) works by: 1. Ignoring small items of size $< \epsilon$. 2. Grouping the remaining n' large items into $k = 1/\epsilon^2$ groups, each of size $q = \lfloor n'\epsilon^2 \rfloor$. 3. Rounding up the size of each item in Group i to the maximum in that group. Let I' be the rounded instance. Because

we rounded up, any packing for I' is valid for the original large items. By discarding the first group (which has size $q \leq n'\epsilon^2 \leq \text{OPT}\epsilon$), we can map the packing of Group i to the slots of Group $i - 1$ in the optimal packing. This yields:

$$\text{OPT}(I') \leq \text{OPT}(I) + q \leq (1 + \epsilon)\text{OPT}(I)$$

Packing the discarded group adds at most $q \leq 1/\epsilon^2$ bins. Finally, packing the small items using First-Fit either opens no new bins (keeping the size within $(1 + \epsilon)\text{OPT} + O(1/\epsilon^2)$) or, if it does, all bins except the last are packed to at least $1 - \epsilon$ density, yielding an overall $(1 + \epsilon)\text{OPT} + 1$ guarantee.

9.5 Applications

- **Cloud Computing (VM Allocation):** Allocating virtual machines (VMs) of different CPU/memory demands (items) to physical servers of unit capacity (bins) to minimize server leasing costs.
- **Logistics and Container Loading:** Packing boxes of different weights into standard shipping containers to minimize shipping costs.
- **Ad Placement:** Fitting ads of different durations into standard ad slots.

Real Example: Virtual Machine Allocation in Cloud Data Centers

Scenario: A cloud provider (e.g. AWS) receives requests to run $n = 5,000$ virtual machines of various sizes (e.g. 0.1, 0.25, 0.4 of a physical server's resources). The provider wants to host them on the minimum number of physical servers to save energy. **Model:** Standard Bin Packing. Physical servers are unit-capacity bins. Items are VM resource demands. First-Fit Decreasing and APTAS find the near-optimal VM-to-server allocation. **Why FFD and APTAS matter:** Energy cost in data centers is a major expense. First-Fit Decreasing is offline and achieves a tight 11/9-approximation in practice. The APTAS allows trading off computation time for an arbitrarily close-to-optimal packing, saving millions of dollars in infrastructure costs.

9.6 Tight Example: Next-Fit Heuristic

A tight example for Next-Fit consists of $2n$ items of alternating sizes $0.5 + \delta$ and $0.5 - \delta$ for a small $\delta > 0$. Next-Fit will place each pair in a new bin because $0.5 + \delta + 0.5 - \delta = 1.0$, but if the next item is $0.5 + \delta$, it cannot fit in the current bin of remaining capacity $0.5 - \delta$. Next-Fit will use $2n$ bins, whereas the optimal packing can pair each $0.5 + \delta$ with a $0.5 - \delta$, using only n bins. The ratio approaches 2.

9.7 Exercises

Exercise 9.1. Prove that the number of bins used by Next-Fit is at most $2 \cdot \text{OPT} + 1$.

Exercise 9.2. Prove that if all item sizes are strictly greater than $1/3$, then the First-Fit Decreasing algorithm finds an optimal packing.

Chapter 10

Minimum Makespan Scheduling

10.1 Intuition: Load Balancing and Dual Approximation

Intuition 10.1. Peeling and Layering Intuition: The goal of Minimum Makespan Scheduling is to allocate n jobs on m identical machines to minimize the maximum machine completion time (makespan). A simple online greedy rule (Graham's List Scheduling) achieves a 2-approximation by assigning each job to the least loaded machine. Sort-based greedy (LPT rule) improves this to a $4/3$ -approximation by scheduling larger jobs first. For a PTAS, we use Hochbaum-Shmoys dual approximation: we guess the makespan T , round large job sizes down to reduce the distinct sizes to a constant, pack them using DP, and then place small jobs greedily. Binary searching T yields a $(1 + \epsilon)$ schedule.

10.2 Problem Definition

Given n jobs with processing times $p_1, \dots, p_n \geq 0$ and m identical parallel machines, the *Minimum Makespan Scheduling* problem is to assign each job to a machine such that the maximum total processing time of any machine is minimized.

The problem is NP-hard. It admits a Polynomial-Time Approximation Scheme (PTAS), which finds a schedule of makespan at most $(1 + \epsilon)\text{OPT}$ in time polynomial in n for any fixed $\epsilon > 0$.

10.3 Algorithm and Python Implementation

We implement Graham's List Scheduling, the LPT heuristic, and the Hochbaum-Shmoys PTAS using dual approximation via bin packing.

```
1 def makespan_ptas(jobs, m, eps=0.25):
2     """Hochbaum-Shmoys PTAS for Minimum Makespan Scheduling."""
3     lb = max(max(jobs), sum(jobs) / m)
4     ub = max(max(jobs), 2 * sum(jobs) / m)
5
6     best_schedule = None
7     for _ in range(15):
8         mid = (lb + ub) / 2
9         success, schedule = check_schedule(jobs, m, mid, eps)
```



```

10         if success:
11             best_schedule = schedule
12             ub = mid
13         else:
14             lb = mid
15
16     if not best_schedule:
17         _, best_schedule = check_schedule(jobs, m, ub, eps)
18     return best_schedule

```

Listing 10.1: Hochbaum-Shmoys PTAS for Minimum Makespan

10.4 Analysis: LPT and Hochbaum-Shmoys PTAS

Graham’s List Scheduling achieves an approximation factor of $2 - 1/m$. The LPT heuristic achieves a factor of $4/3 - 1/(3m)$ by sorting jobs.

The PTAS uses dual approximation: instead of minimizing makespan directly, it solves the dual decision problem: “Can we schedule all jobs within makespan T ?” 1. Jobs of size $\leq \epsilon T$ are “small” and can be placed greedily. 2. Jobs of size $> \epsilon T$ are “large.” We round each large job down to the nearest multiple of $\epsilon^2 T$. This yields at most $1/\epsilon^2$ distinct large sizes. 3. We test if the rounded large jobs can be packed into m bins of capacity T using dynamic programming. If they can, the rounding error adds at most $(1/\epsilon) \times \epsilon^2 T = \epsilon T$ to the load of any machine, giving a makespan $\leq (1 + \epsilon)T$ for the original large jobs. Placing small jobs keeps the makespan $\leq (1 + \epsilon)T$ or proves $T < \text{OPT}$. Binary searching T in $[LB, UB]$ yields the $(1 + \epsilon)$ PTAS.

10.5 Applications

- **CPU / GPU Task Scheduling:** Assigning threads or compute tasks to identical CPU/GPU cores to minimize total execution time (latency).
- **Manufacturing:** Allocating identical production lines to process a batch of customer orders.
- **Distributed Computing MapReduce:** Distributing map or reduce tasks to cluster nodes.

Real Example: Multicore CPU Load Balancing

Scenario: An OS scheduler has $n = 50$ computational tasks with known durations (e.g. 5ms, 12ms, 30ms) to run on a quad-core CPU ($m = 4$). Goal: distribute tasks to minimize total execution time. **Model:** Minimum Makespan Scheduling. Tasks are jobs, CPU cores are identical machines. FFD/LPT or PTAS provides the optimal scheduling. **Why PTAS matters:** Maximizing CPU utilization is key to overall system performance. A PTAS guarantees that the multicore scheduling is within ϵ of the theoretical minimum latency possible, which avoids core idling and maximizes throughput.

10.6 Tight Example: LPT Heuristic

A tight example for LPT on 2 machines consists of 5 jobs of sizes 3, 3, 2, 2, 2. LPT sorts them and schedules the two 3s on separate machines, then places the 2s, yielding a schedule of loads

$3 + 2 + 2 = 7$ and $3 + 2 = 5$. The makespan is 7. The optimal schedule pairs a 3 and a 2 on each machine, and places the remaining 2 on one of them, achieving loads $3 + 2 = 5$ and $3 + 2 + 2 = 7$? No, optimal is loads $3 + 3 = 6$ and $2 + 2 + 2 = 6$, makespan 6. The ratio is $7/6 \approx 1.167$, which matches $4/3 - 1/(3m) = 4/3 - 1/6 = 7/6$.

10.7 Exercises

Exercise 10.1. *Prove that Graham's List Scheduling achieves an approximation factor of exactly $2 - 1/m$.*

Exercise 10.2. *Construct a tight example for LPT scheduling on $m = 3$ machines showing the ratio is $4/3 - 1/(3m) = 11/9$.*

Chapter 11

Euclidean TSP

11.1 Intuition: Geometric Dissection and Portals

Intuition 11.1. *Peeling and Layering Intuition:* General Metric TSP is hard to approximate below 1.5. However, if vertices are points in a 2D Euclidean plane, we can use the geometric structure to construct a PTAS. Arora's PTAS works by enclosing all points in a bounding box, applying a randomized shift, and recursively partitioning the box into a quadtree. By restricting the tour to cross cell boundaries only at a small number of designated points called portals, the problem can be solved via dynamic programming. We implement a divide-and-conquer version of this quadtree partition: solving sub-quadrants recursively and merging tours using cheapest 2-edge connections.

11.2 Problem Definition

Given n points in the Euclidean plane $\mathbb{R}^2$, the *Euclidean Traveling Salesman Problem* is to find a closed tour of minimum total Euclidean length that visits every point exactly once.

The problem remains NP-hard. However, unlike general Metric TSP, it admits a Polynomial-Time Approximation Scheme (PTAS) that achieves a $(1 + \epsilon)$ -approximation in $n^{O(1/\epsilon)}$ time.

11.3 Algorithm and Python Implementation

We implement the exact Held-Karp dynamic programming baseline, alongside the quadtree divide-and-conquer cycle merging heuristic which serves as the geometric partitioning engine.

```
1 def merge_tours(tour1, tour2, points):
2     """Cheapest connection swap between two tours or insertion of a single
3     point."""
4     if not tour1: return tour2
5     if not tour2: return tour1
6     dist = lambda u, v: math.hypot(points[u][0]-points[v][0], points[u]
7         [1]-points[v][1])
8
9     # Case 1 & 2: Single point insertion
10    if len(tour2) <= 2:
```

```

9      u = tour2[0]
10     best_diff, best_idx = float('inf'), -1
11     for i in range(len(tour1) - 1):
12         diff = dist(tour1[i], u) + dist(u, tour1[i+1]) - dist(tour1[i],
13             tour1[i+1])
14         if diff < best_diff:
15             best_diff, best_idx = diff, i
16     return tour1[:best_idx + 1] + [u] + tour1[best_idx + 1:]
17 if len(tour1) <= 2:
18     return merge_tours(tour2, tour1, points)
19
20 # Case 3: Merge two closed cycles
21 t1, t2 = list(tour1), list(tour2)
22 n1, n2 = len(t1) - 1, len(t2) - 1
23 best_diff, best_swap = float('inf'), None
24
25 for i in range(n1):
26     for j in range(n2):
27         diff1 = dist(t1[i], t2[j]) + dist(t1[i+1], t2[j+1]) - dist(t1[i],
28             t1[i+1]) - dist(t2[j], t2[j+1])
29         diff2 = dist(t1[i], t2[j+1]) + dist(t1[i+1], t2[j]) - dist(t1[i],
30             t1[i+1]) - dist(t2[j], t2[j+1])
31         if diff1 < best_diff: best_diff, best_swap = diff1, (i, j,
32             False)
33         if diff2 < best_diff: best_diff, best_swap = diff2, (i, j,
34             True)
35
36 i, j, reverse = best_swap
37 t2_rot = t2[j:-1] + t2[:j]
38 if reverse: t2_rot.reverse()
39 return t1[:i+1] + t2_rot + t1[i+1:]

```

Listing 11.1: Cheapest swap to merge tours

11.4 Analysis: Arora's PTAS and Portals

Arora's PTAS achieves a $(1 + \epsilon)$ -approximation by proving that:

1. **Structure Theorem:** There exists a near-optimal tour (cost $\leq (1 + \epsilon)\text{OPT}$) that crosses the boundary of each quadtree square at most $r = O(1/\epsilon)$ times, and only at designated *portals* spaced along the square's perimeter.
2. **Randomized Shifting:** By shifting the quadtree coordinates randomly, we ensure that the probability of a bounding line intersecting a dense region of the tour (and thus requiring many crossings) is small. The expected cost added by routing the tour through portals is at most $\epsilon \cdot \text{OPT}$.
3. **Dynamic Programming:** For each quadtree cell, the number of portal configurations is small (at most $(O(1/\epsilon \log L))^{O(1/\epsilon)}$). We can find the optimal set of paths within each cell using dynamic programming, proceeding from the leaves of the quadtree up to the root.

11.5 Applications

- **PCB Drilling and Tool-Path Optimization:** Minimizing the travel distance of a drill bit moving across coordinates on a circuit board.

- **Robotics and Automated Navigation:** Path planning for automated warehouse forklifts or domestic vacuum cleaners.
- **Neighborhood Package Delivery:** Local routing for delivery trucks within a localized zip code.

Real Example: High-Speed PCB Drilling Optimization

Scenario: A manufacturing line needs to drill $n = 10,000$ micro-vias on a silicon wafer. The drilling head moves in 2D space. The total production throughput is directly limited by the travel time of the drill head between via locations. **Model:** Euclidean TSP. Drill coordinates are points in 2D space. A fast, high-quality tour is required to minimize motor wear and cycle time. **Why PTAS matters:** Even a 5% reduction in tour distance translates to thousands of dollars saved daily in high-volume electronics manufacturing. By employing a quadtree-based PTAS, we partition the drill grid recursively, optimize clusters locally, and merge them into an overall near-optimal toolpath.

11.6 Tight Example: Shifting Bottleneck

If we do not shift the quadtree boundaries, a simple configuration of points can yield a poor approximation. Consider two parallel rows of points placed very close to one another, but separated by the root division line $x = x_{mid}$. Without shifting, the algorithm is forced to solve the left half and right half independently and then merge them. The merge step must connect across the boundary line, forcing a long loop that increases the cost, whereas a simple zigzag pattern crossing the boundary line many times would be optimal. Shifting the quadtree randomly ensures that with high probability, the main split line does not separate these tightly coupled rows.

11.7 Exercises

Exercise 11.1. *Prove that for a quadtree cell of width W , if we place m equally spaced portals on each boundary line, the distance from any point on a boundary to its nearest portal is at most $W/(2m)$.*

Exercise 11.2. *Explain why the number of ways to match $2k$ portals into k disjoint paths is $(2k)!/(2^k k!)$.*

Chapter 12

Introduction to LP-Duality

12.1 Intuition: Two Sides of the Same Coin

Intuition 12.1. Primal-Dual Intuition: Every minimization problem has a maximization “dual” that provides lower bounds. The primal variables are the algorithm’s decisions (which sets to pick, which vertices to cover). The dual variables are prices or charges assigned to constraints (elements to cover, edges to cut).

Weak Duality: Any feasible dual solution gives a lower bound on the primal optimum. This is the engine of approximation algorithms — we construct primal and dual solutions together, ensuring the primal cost is within a constant factor of the dual value.

12.2 The LP-Duality Theorem

Theorem 12.1 (LP Duality). Let primal (P) be $\min\{c^T x : Ax \geq b, x \geq 0\}$ and dual (D) be $\max\{b^T y : A^T y \leq c, y \geq 0\}$. Then:

1. **Weak Duality:** $c^T x \geq b^T y$ for all feasible x, y
2. **Strong Duality:** If (P) has finite optimum, so does (D) , and $\min = \max$
3. **Complementary Slackness:** $x_i^*(c - A^T y^*)_i = 0$ and $y_j^*(Ax^* - b)_j = 0$

12.3 Min-Max Relations as LP Duality

Classic theorems are LP duality in disguise:

- **Max-flow Min-cut:** Primal = max flow, Dual = min cut
- **König-Egerváry:** Primal = max matching, Dual = min vertex cover (bipartite)
- **Menger’s Theorem:** Max disjoint paths = Min separator

12.4 Two Fundamental Algorithm Design Techniques

1. **Rounding (Chapters 14, 16-21):** Solve LP relaxation, round fractional x^* to integer $\hat{x}$. Approximation factor $\leq$ integrality gap.
2. **Primal-Dual Schema (Chapters 13, 22-23):** Construct primal and dual solutions simultaneously. Start with dual $y = 0$, increase until a constraint becomes tight, then add corresponding primal variable.

12.4.1 Integrality Gap

The integrality gap of an LP relaxation is $\sup \frac{IP_{opt}}{LP_{opt}}$. No rounding-based algorithm can beat the integrality gap.

12.5 Python: Simple Simplex Solver

```

1 class Simplex:
2     """Maximize c^T x s.t. Ax <= b, x >= 0"""
3     def __init__(self, A, b, c):
4         self.m, self.n = len(A), len(A[0])
5         self.A, self.b, self.c = A, b, c
6
7     def solve(self):
8         # Add slack variables, build tableau
9         tab = [self.A[i] + [1 if j==i else 0 for j in range(self.m)] + [
10             self.b[i]]
11             for i in range(self.m)]
12         tab.append([-ci for ci in self.c] + [0]*self.m + [0])
13
14         while True:
15             # Entering variable (most negative in obj row)
16             entering = min(range(self.n + self.m), key=lambda j: tab[-1][j])
17             if tab[-1][entering] >= -1e-9: break
18
19             # Leaving variable (min ratio test)
20             leaving = min(range(self.m),
21                           key=lambda i: tab[i][-1]/tab[i][entering] if tab[i][
22                               entering] > 1e-9 else float('inf'))
23             if tab[leaving][entering] <= 1e-9: return None, float('inf')
24
25             # Pivot
26             piv = tab[leaving][entering]
27             tab[leaving] = [v/piv for v in tab[leaving]]
28             for i in range(self.m + 1):
29                 if i != leaving:
30                     factor = tab[i][entering]
31                     tab[i] = [tab[i][j] - factor*tab[leaving][j] for j in

```

```
32     for i in range(self.m):
33         for j in range(self.n + self.m):
34             if abs(tab[i][j] - 1) < 1e-9 and all(abs(tab[r][j]) < 1e-9
35                 for r in range(self.m+1) if r != i):
36                 if j < self.n: x[j] = tab[i][-1]
37     return x, tab[-1][-1]
```

Listing 12.1: Simplex for Small LPs

12.6 Exercises

Exercise 12.2. *Write the dual of the Set Cover LP and interpret the variables as prices.*

Exercise 12.3. *Show that the integrality gap of the Set Cover LP is H_n .*

Chapter 13

Set Cover via Dual Fitting

13.1 Intuition: Prices as Dual Variables

Intuition 13.1. Dual Fitting: The greedy algorithm assigns a price to each element when it's covered. These prices don't form a feasible dual solution (they may violate $\sum_{e \in S} \text{price}(e) \leq c(S)$). But if we scale all prices down by H_n , they do become feasible! This proves the H_n bound without explicitly solving the LP.

13.2 Dual of Set Cover LP

Primal: $\min \sum_S c(S) x_S$ s.t. $\sum_{S \ni e} x_S \geq 1 \ \forall e, x_S \geq 0$
Dual: $\max \sum_e y_e$ s.t. $\sum_{e \in S} y_e \leq c(S) \ \forall S, y_e \geq 0$

13.2.1 Dual-Fitting Analysis of Greedy

Theorem 13.1. The prices y_e assigned by the greedy algorithm, scaled by $1/H_n$, form a feasible dual solution.

Proof. When element e is covered, there are k uncovered elements. Its price $\leq \text{OPT}/k$. Summing over all elements in any set S : at most $\text{OPT} \cdot H_n$. Scaling by $1/H_n$ makes dual feasible. $\square$

13.2.2 Can the Guarantee Be Improved?

Feige (1998): No polynomial-time algorithm achieves $(1 - \epsilon) \ln n$ unless $\text{NP} \subseteq \text{DTIME}(n^{\log \log n})$. Greedy is essentially optimal.

13.3 Generalizations

13.3.1 Constrained Set Multicover

Each element e has requirement r_e . Greedy with H_{nr} bound where $r = \max r_e$.

13.3.2 Dual Fitting for Multicover

Prices assigned per unit of coverage. Same analysis extends.

13.4 Other Algorithms for Set Cover

Table 13.1: Set Cover Algorithms

Algorithm	Approximation	Technique
Greedy	H_n	Combinatorial
LP Rounding (Ch. 14)	f (max frequency)	LP Rounding
Primal-Dual (Ch. 22)	f	Primal-Dual Schema
Randomized Rounding	$O(\log n)$	LP + Randomization

13.5 Python Implementation

```

1 def set_cover_primal_dual(universe, sets, costs):
2     """f-approx for Set Cover via Primal-Dual."""
3     covered = set()
4     y = {e: 0.0 for e in universe}
5     picked = []
6
7     # Precompute sets containing each element
8     sets_containing = {e: [] for e in universe}
9     for s, se in sets.items():
10         for e in se:
11             if e in universe:
12                 sets_containing[e].append(s)
13
14     while covered != universe:
15         e = next(iter(universe - covered))
16
17         # Raise y[e] until some set becomes tight
18         while True:
19             min_slack = float('inf')
20             tight_set = None
21             for s in sets_containing[e]:
22                 slack = costs[s] - sum(y[ee] for ee in sets[s] if ee in
23                                     universe)
24                 if slack < min_slack:
25                     min_slack = slack
26                     tight_set = s
27
28             if min_slack <= 1e-9: break
29             y[e] += min_slack
30
31             for s in sets_containing[e]:
32                 slack = costs[s] - sum(y[ee] for ee in sets[s] if ee in
33                                     universe)
34                 if abs(slack) < 1e-9:
35                     tight_set = s
36                     break
37             if tight_set: break

```

```
37         if tight_set and tight_set not in picked:
38             picked.append(tight_set)
39             covered.update(sets[tight_set])
40
41     return picked
```

Listing 13.1: Primal-Dual Set Cover (f-approx)

13.6 Exercises

Exercise 13.2. *Show greedy set cover achieves H_k where $k = \max |S_i|$.*

Exercise 13.3. *Give a factor- f algorithm where f is maximum element frequency.*

Chapter 14

Rounding Applied to Set Cover

14.1 Intuition: Threshold Rounding

Intuition 14.1. LP Rounding: Solve the LP relaxation, get fractional x_S^* . Pick all sets with $x_S^* \geq 1/f$ where f is maximum element frequency. Since each element appears in at most f sets, and $\sum_{S \ni e} x_S^* \geq 1$, at least one set containing e must have $x_S^* \geq 1/f$. So e is covered!

14.2 LP Relaxation for Set Cover

$$\min \sum_S c(S)x_S \quad \text{s.t.} \quad \sum_{S \ni e} x_S \geq 1 \quad \forall e, \quad x_S \geq 0$$

Algorithm 14.1 (Set Cover via LP Rounding — Factor f). 1. Solve LP relaxation, get optimal x^*

2. Let $f = \max_e |\{S : e \in S\}|$ (maximum frequency)

3. Pick all sets with $x_S^* \geq 1/f$

Theorem 14.2. This gives an f -approximation for Set Cover.

Proof. For any e , $\sum_{S \ni e} x_S^* \geq 1$. With at most f terms, at least one $x_S^* \geq 1/f$, so e is covered. Cost $\leq f \sum c(S)x_S^* = f \cdot \text{OPT}_{LP} \leq f \cdot \text{OPT}_{IP}$. $\square$

14.2.1 Vertex Cover via LP Rounding (Factor 2)

Vertex Cover is Set Cover with $f = 2$ (each edge in exactly 2 sets). LP rounding gives a clean 2-approximation.

Algorithm 14.3 (Vertex Cover via LP Rounding — Factor 2). 1. Solve LP: $\min \sum_v x_v$ s.t. $x_u + x_v \geq 1 \quad \forall (u, v) \in E, x_v \geq 0$

2. Pick all vertices with $x_v \geq 1/2$

This matches the combinatorial 2-approximation from Chapter 1 but via LP duality.

14.2.2 Integrality Gap

The integrality gap of the Set Cover LP is H_n (tight example from Chapter 2). So no rounding-based algorithm can beat H_n .

14.3 Python Implementation

```

1 def set_cover_lp_rounding(universe, sets, costs):
2     """f-approx for Set Cover via LP Rounding."""
3     x, opt = set_cover_lp(universe, sets, costs)
4     if not x: return [], 0.0
5
6     S = list(sets.keys())
7     f = max(sum(1 for s in S if e in sets[s]) for e in universe)
8     threshold = 1.0 / f
9
10    picked = [s for i, s in enumerate(S) if x[i] >= threshold]
11    total_cost = sum(costs[s] for s in picked)
12    return picked, total_cost
13
14 def vertex_cover_lp_rounding(graph):
15     """2-approx for Vertex Cover via LP Rounding."""
16     # Map to Set Cover to solve via Dual LP
17     edges = [(u,v) for u in graph for v in graph[u] if u < v]
18     universe = set(range(len(edges)))
19     edge_to_idx = {e: i for i, e in enumerate(edges)}
20
21     vertices = list(graph.keys())
22     sets = {v: {edge_to_idx[(min(v,w), max(v,w))]} for w in graph[v]}
23             for v in vertices}
24     costs = {v: 1.0 for v in vertices}
25
26     x, opt = set_cover_lp(universe, sets, costs)
27     return {vertices[i] for i in range(len(vertices)) if x[i] >= 0.5}

```

Listing 14.1: LP Rounding for Set Cover

14.4 Exercises

Exercise 14.4. Show that the integrality gap of the Set Cover LP is H_n .

Exercise 14.5. Prove that the LP rounding for Vertex Cover gives a 2-approximation.

Chapter 15

Weighted Vertex Cover via Primal-Dual

15.1 Intuition: The Classical Schema

Intuition 15.1. *Peeling and Layering Intuition:* The primal-dual schema is one of the most powerful paradigms for designing approximation algorithms. Starting with a feasible dual solution (initially set to zero) and an infeasible primal solution (initially empty), the algorithm iteratively increases the dual variables of unsatisfied constraints until one or more dual constraints become tight. The tight constraints dictate which primal variables should be set. This ensures that we construct a primal cover whose cost is bounded by a constant factor of the dual objective value (which serves as a lower bound on the optimal primal cost).

15.2 Problem Definition

Given an undirected graph $G = (V, E)$ with non-negative vertex weights $w_v \geq 0$ for each $v \in V$, the *Weighted Vertex Cover* problem is to find a subset of vertices $U \subseteq V$ of minimum total weight such that every edge in E has at least one endpoint in U .

The problem is NP-hard. We seek a 2-approximation using the LP primal-dual schema.

15.3 Algorithm and Python Implementation

We implement the primal-dual schema: we initialize $y_e = 0$ for all $e \in E$, and for any uncovered edge, we raise its dual variable y_e until a vertex becomes tight.

```
1 def vertex_cover_primal_dual(vertices, edges, weights):
2     """Primal-Dual 2-approximation for Weighted Vertex Cover."""
3     y = {(min(u, v), max(u, v)): 0.0 for u, v in edges}
4     vertex_dual_sums = {v: 0.0 for v in vertices}
5     cover = set()
6
7     for u, v in edges:
8         e = (min(u, v), max(u, v))
9         if u not in cover and v not in cover:
10             slack_u = weights[u] - vertex_dual_sums[u]
11             slack_v = weights[v] - vertex_dual_sums[v]
```

```

12         raise_amount = min(slack_u, slack_v)
13
14         y[e] += raise_amount
15         vertex_dual_sums[u] += raise_amount
16         vertex_dual_sums[v] += raise_amount
17
18         if abs(vertex_dual_sums[u] - weights[u]) < 1e-9:
19             cover.add(u)
20         if abs(vertex_dual_sums[v] - weights[v]) < 1e-9:
21             cover.add(v)
22     return cover, y

```

Listing 15.1: Primal-Dual Schema for Weighted Vertex Cover

15.4 Analysis: Factor 2 Approximation

Let the primal LP be:

$$\min \sum_{v \in V} w_v x_v \quad \text{s.t.} \quad x_u + x_v \geq 1 \quad (\forall (u, v) \in E), \quad x_v \geq 0$$

The dual LP is:

$$\max \sum_{e \in E} y_e \quad \text{s.t.} \quad \sum_{e \ni v} y_e \leq w_v \quad (\forall v \in V), \quad y_e \geq 0$$

By construction, our algorithm ensures that for any selected vertex $v \in \text{cover}$, its dual constraint is tight: $\sum_{e \ni v} y_e = w_v$. Thus, the total weight of the selected cover is:

$$\sum_{v \in \text{cover}} w_v = \sum_{v \in \text{cover}} \sum_{e \ni v} y_e \leq 2 \sum_{e \in E} y_e$$

where the factor of 2 arises because each edge $e = (u, v)$ can have at most both of its endpoints in the cover. By weak duality, $\sum_{e \in E} y_e \leq \text{OPT}$, which guarantees that:

$$\text{Cost}(\text{cover}) \leq 2 \cdot \text{OPT}$$

This proves that the primal-dual algorithm is a 2-approximation.

15.5 Applications

- **Network Security:** Minimizing the cost of monitoring nodes to cover all communication links (edges).
- **Resource Optimization:** Selecting the cheapest routers/servers to cover all transmission paths.
- **Facilities Protection:** Placing guards at vertices of a street network to observe all streets.

Real Example: Network Traffic Monitoring with Cost Constraints

Scenario: An ISP wants to monitor all optical fiber links (edges) connecting a set of routers (vertices). Placing monitoring software on router v costs w_v . The ISP wants to cover all links while minimizing the software licensing cost. **Model:** Weighted Vertex Cover. Routers are vertices, fiber links are edges. Primal-Dual algorithm finds the near-optimal router subset to monitor. **Why Primal-Dual matters:** LP solvers can be computationally expensive and may return fractional values requiring complex rounding. The primal-dual schema runs in linear time $O(|V| + |E|)$ and provides a fast, integer, and guaranteed factor 2 solution, which is ideal for real-time traffic monitoring systems.

15.6 Tight Example: Star Graph and Complete Bipartite Graph

Consider a star graph with center vertex 0 and leaf vertices $1, 2, \dots, k$. Let the center have weight $w_0 = k$, and each leaf have weight $w_i = 1$. The optimal vertex cover is $\{0\}$ with cost k . If we process the edges $(0, 1), (0, 2), \dots, (0, k)$ in order, the primal-dual algorithm will raise $y_{(0,i)}$ to 1.0 for each edge, tightening the leaves $1, \dots, k$ and the center vertex 0 simultaneously. The algorithm can include both the center and all leaves in the cover, yielding a total cost of $k + k = 2k$. The ratio is $2k/k = 2$, showing the factor 2 bound is tight.

15.7 Exercises

Exercise 15.1. *Prove that the primal-dual vertex cover algorithm is exactly equivalent to the local-ratio vertex cover algorithm (which scales vertex weights by subtraction).*

Exercise 15.2. *Construct a tight example for the primal-dual algorithm on a path graph of length 5 with vertex weights 1, 1, 1, 1, 1.*

Chapter 16

Randomized Rounding

16.1 Intuition: LP Probabilistic Rounding

Intuition 16.1. Peeling and Layering Intuition: LP rounding algorithms are often deterministic (e.g. Set Cover rounding at threshold $1/f$). However, for problems like Max-SAT, deterministic thresholding can be suboptimal. Instead, we can interpret the fractional LP variables y_i^* as probabilities. By setting variable x_i to True with probability y_i^* , we ensure that the expected value of the assignment is close to the LP optimal value. By combining this with a simple coin-flip assignment (which performs well on large clauses, while LP rounding performs well on small clauses), we obtain a $3/4$ -approximation.

16.2 Problem Definition

Given n boolean variables $x_1, \dots, x_n$ and m clauses $C_1, \dots, C_m$ with weights $w_j \geq 0$, the *Maximum Satisfiability* (Max-SAT) problem is to find a truth assignment to the variables that maximizes the total weight of satisfied clauses.

The decision version of SAT is NP-complete. Max-SAT is NP-hard. We seek a randomized $(1 - 1/e)$ -approximation (approx. 0.632) via LP rounding, and a $3/4$ -approximation (0.75) by choosing the best of LP rounding and random coin-flipping.

16.3 Algorithm and Python Implementation

We formulate the Max-SAT problem as an integer program, solve its LP relaxation using our Simplex solver, and perform randomized rounding.

```
1 def solve_max_sat_lp(n_vars, clauses, weights):
2     """Formulate and solve LP relaxation for Max-SAT."""
3     n_cols = n_vars + len(causes)
4     A, b = [], []
5
6     # Clause constraints:  $z_j - \sum_{i \in C_j^+} y_i + \sum_{i \in C_j^-} y_i \leq |C_j^-|$ 
7     for j, (pos, neg) in enumerate(causes):
8         row = [0.0] * n_cols
```

```

9         row[n_vars + j] = 1.0
10        for i in pos: row[i] = -1.0
11        for i in neg: row[i] = 1.0
12        A.append(row)
13        b.append(float(len(neg)))
14
15        # Bounds: y_i <= 1, z_j <= 1
16        for i in range(n_vars):
17            row = [0.0] * n_cols; row[i] = 1.0; A.append(row); b.append(1.0)
18        for j in range(len(clauses)):
19            row = [0.0] * n_cols; row[n_vars + j] = 1.0; A.append(row); b.
20                append(1.0)
21
22        c = [0.0] * n_cols
23        for j in range(len(clauses)): c[n_vars + j] = weights[j]
24
25        solver = Simplex(A, b, c)
26        sol, obj = solver.solve()
27        return sol[:n_vars], sol[n_vars:], obj
28
29    def randomized_rounding_max_sat(n_vars, clauses, weights, y_lp, trials
30    =200):
31        """Applies randomized rounding to LP variable values."""
32        best_weight, best_assignment, total_weight = -1.0, [], 0.0
33        for _ in range(trials):
34            assignment = [random.random() < y_lp[i] for i in range(n_vars)]
35            weight = evaluate_assignment(assignment, clauses, weights)
36            total_weight += weight
37            if weight > best_weight:
38                best_weight, best_assignment = weight, assignment
39        return best_assignment, total_weight / trials, best_weight

```

Listing 16.1: Randomized Rounding for Max-SAT

16.4 Analysis: Approximations and Probability

For a clause C_j with size k , let its index set of positive and negative literals be C_j^+ and C_j^- . The LP relaxation includes:

$$z_j \leq \sum_{i \in C_j^+} y_i + \sum_{i \in C_j^-} (1 - y_i)$$

Under randomized rounding, variable x_i is set to True with probability y_i^* . The probability that clause C_j is satisfied is:

$$1 - \prod_{i \in C_j^+} (1 - y_i^*) \prod_{i \in C_j^-} y_i^*$$

Using the arithmetic-geometric mean inequality, we can show this probability is at least:

$$1 - \left(1 - \frac{z_j}{k}\right)^k$$

Since $g(z) = 1 - (1 - z/k)^k$ is concave and $g(0) = 0, g(1) = 1 - (1 - 1/k)^k$, we have:

$$\Pr[C_j \text{ is satisfied}] \geq \left[1 - \left(1 - \frac{1}{k}\right)^k\right] z_j \geq \left(1 - \frac{1}{e}\right) z_j$$

Summing over all clauses, the expected weight satisfied is $\geq (1 - 1/e) \cdot \text{OPT}_{LP} \approx 0.632 \cdot \text{OPT}$. For small clauses, this bound is much higher (e.g. $k = 1$ gives $1 \cdot z_j$, $k = 2$ gives $0.75 \cdot z_j$). For large clauses, the coin-flip baseline satisfies them with probability $1 - 2^{-k}$. Choosing the best of both algorithms yields a 3/4-approximation (Goemans-Williamson 1994).

16.5 Applications

- **Satisfiability Solvers:** Finding satisfying assignments for weighted CNF formulas in hardware design verification.
- **Database Query Optimizers:** Selecting indexing configurations to satisfy as many query filters as possible under physical memory limits.
- **Digital Circuit Testing:** Generating test vectors that cover maximum gate combinations.

Real Example: Database Query Filter Maximization

Scenario: A high-speed database system (e.g., Google Bigtable) has a set of query requests. Each query is a set of boolean filters (clauses). We want to index a subset of fields (variables) to accelerate the queries. If we index a field, it satisfies positive/negative constraints. We want to maximize queries served. **Model:** Max-SAT. Variables are index decisions, clauses are query filters. Randomized LP rounding finds the indexing strategy that maximizes query throughput. **Why Randomized Rounding matters:** Real systems cannot wait for exact integer programming solvers. Randomized LP rounding runs instantly by solving a continuous linear program and sampling variable assignments, guaranteeing that query satisfaction is within a fraction of the absolute optimal configuration.

16.6 Tight Example: Clause Size Bottleneck

A tight example for the $(1 - 1/e)$ bound consists of n independent single-literal clauses of the form (x_i) for $i = 1, \dots, n$, each with weight 1. The LP relaxation will set $y_i^* = 1.0$ and $z_i^* = 1.0$, achieving objective n . Randomized rounding sets each x_i to True with probability $y_i^* = 1.0$, satisfying all clauses exactly. Wait! What if we have a clause $(x_1 \vee x_2 \vee \dots \vee x_k)$? As k grows, the factor $1 - (1 - 1/k)^k$ converges to $1 - 1/e \approx 0.632$. This shows the asymptotic bound is tight for large clauses under pure randomized rounding, but the coin-flip algorithm achieves $1 - 2^{-k} \approx 1$ for large clauses, which compensates for this bottleneck.

16.7 Exercises

Exercise 16.1. Prove that for any clause C_j of size k , the probability that it is satisfied by the coin-flip assignment is $1 - 2^{-k}$.

Exercise 16.2. Show that if all clauses in a Max-SAT instance have size 1, the randomized rounding algorithm finds an exact optimal solution.

Chapter 17

Chernoff Bounds

17.1 Intuition: Concentration and Coverage

Intuition 17.1. Peeling and Layering Intuition: In many randomized algorithms, we want to prove that the sum of independent random variables is close to its expected value. This is achieved by Chernoff bounds, which show that the tails of the distribution decay exponentially. When applying randomized rounding to Set Cover, we select set S with probability x_S^* . To ensure all elements are covered, we run this selection $O(\log n)$ times. Chernoff bounds allow us to prove that the probability of failing to cover any element is extremely small, and the total cost remains within $O(\log n) \cdot OPT$.

17.2 Problem Definition

Given a universe U of n elements and a collection of sets $\mathcal{S}$ with costs c_S , the *Set Cover* problem is to find a minimum cost subcollection of $\mathcal{S}$ that covers U .

The problem is NP-hard. We seek a randomized $O(\log n)$ -approximation that succeeds with high probability $(1 - 1/n)$.

17.3 Algorithm and Python Implementation

We solve the LP relaxation of Set Cover and perform multi-round randomized rounding, repeating the selection process $t = c \ln n$ times.

```
1 def set_cover_randomized_rounding(universe, sets, costs, x_lp, c_factor
   =1.5):
2     """Set Cover randomized rounding using multi-round selection."""
3     n_elements = len(universe)
4     t = int(math.ceil(c_factor * math.log(max(2, n_elements))))
5     chosen_sets = set()
6
7     # Run t independent selection rounds
8     for _ in range(t):
9         for s_idx in range(len(sets)):
10             if random.random() < x_lp[s_idx]:
```

```

11         chosen_sets.add(s_idx)
12
13     covered = set()
14     for s_idx in chosen_sets:
15         covered.update(sets[s_idx])
16     cost = sum(costs[s_idx] for s_idx in chosen_sets)
17     return chosen_sets, cost, (covered == universe)

```

Listing 17.1: Randomized Rounding for Set Cover

17.4 Analysis: Chernoff Bounds and Set Cover

Let x^* be the optimal fractional LP solution. For each element e , the LP constraint is $\sum_{S \ni e} x_S^* \geq 1$. In a single round of randomized rounding, the probability that element e is not covered is:

$$\prod_{S \ni e} (1 - x_S^*) \leq \prod_{S \ni e} e^{-x_S^*} = e^{-\sum_{S \ni e} x_S^*} \leq e^{-1} \leq \frac{1}{e}$$

If we repeat the selection process independently for $t = c \ln n$ rounds, the probability that e remains uncovered is at most:

$$(e^{-1})^t = e^{-c \ln n} = \frac{1}{n^c}$$

Applying the union bound over all n elements, the probability that *any* element is left uncovered is:

$$\Pr[\text{some element is uncovered}] \leq n \cdot \frac{1}{n^c} = \frac{1}{n^{c-1}}$$

For $c = 2$, this probability is at most $1/n$, meaning all elements are covered with probability at least $1 - 1/n$. The expected cost of the chosen sets is:

$$E[\text{Cost}] \leq t \cdot \sum_{S \in \mathcal{S}} c_S x_S^* = O(\log n) \cdot \text{OPT}_{LP}$$

By Markov's inequality, the cost is within $2t \cdot \text{OPT}_{LP}$ with probability at least $1/2$. By running the algorithm a few times, we obtain a valid cover of cost $O(\log n) \cdot \text{OPT}$ with high probability.

17.5 Applications

- **Randomized Load Balancing:** Allocating client queries to servers to prevent congestion and guarantee even CPU distribution.
- **High-Speed Packet Routing:** Selecting paths for network packets to minimize link congestions.
- **Coverage of Wireless Sensor Networks:** Activating sensor groups to cover a target field while maximizing battery life.

Real Example: Query Server Load Balancing

Scenario: A web service (e.g., Netflix) distributes incoming client requests to $m = 100$ parallel servers. Each request demands CPU. We want to distribute them randomly such that no server has CPU load exceeding $1.2\times$ average. **Model:** Randomized load allocation. The sum of task loads on server i is a sum of independent random variables. **Why Chernoff Bounds matter:** Without bounds, a simple random assignment could lead to temporary server crash due to congestion. Chernoff bounds mathematically guarantee that with $n = 10,000$ requests, the probability that any single server becomes overloaded is less than 10^{-6} , ensuring system reliability.

17.6 Tight Example: The Grid Partition

Consider a universe of n elements where sets are structured such that the LP solution is $x_S^* = 1/k$ for k overlapping sets. If we only run 1 round of randomized rounding, each element has a failure probability of $(1 - 1/k)^k \approx 1/e \approx 0.368$. For $n = 1000$ elements, this would leave about 368 elements uncovered. Running $t = c \ln n$ rounds is mathematically required to bring this down to zero.

17.7 Exercises

Exercise 17.1. State the multiplicative Chernoff bound for the sum of independent Bernoulli trials $X = \sum X_i$ with mean μ .

Exercise 17.2. Prove that if we run $t = 3 \ln n$ rounds, the probability that any element remains uncovered is at most $1/n^2$.

Chapter 18

Semidefinite Programming

18.1 Intuition: High-Dimensional Embeddings and Hyperplanes

Intuition 18.1. *Peeling and Layering Intuition:* The Max-Cut problem is NP-hard and cannot be approximated within $16/17 \approx 0.941$ unless $P=NP$. However, we can obtain a 0.878 approximation using Semidefinite Programming (SDP). We model the vertices as unit vectors on a sphere in $\mathbb{R}^n$. Instead of assigning each vertex to 0 or 1, we embed them such that connected vertices are pushed as far apart as possible (minimizing their dot products). To round these vectors, we choose a random hyperplane. Vertices on one side of the hyperplane form one partition, and the rest form the other.

18.2 Problem Definition

Given an undirected graph $G = (V, E)$ with non-negative edge weights $w_{ij} \geq 0$, the *Maximum Cut* (Max-Cut) problem is to find a partition of the vertices V into two sets S and $V \setminus S$ such that the total weight of edges crossing the partition is maximized.

The problem is NP-hard. We seek a randomized 0.878-approximation using the Goemans-Williamson algorithm.

18.3 Algorithm and Python Implementation

We implement a lightweight gradient projection optimizer to find the vector embeddings on the unit sphere and apply randomized hyperplane rounding to partition the vertices.

```
1 def optimize_max_cut_vectors(n, edges, weights, dim=8, lr=0.05, epochs
   =200):
2     """Finds unit vector embeddings in  $\mathbb{R}^{\text{dim}}$  for Max-Cut."""
3     v = []
4     for _ in range(n):
5         vec = [random.gauss(0.0, 1.0) for _ in range(dim)]
6         mag = math.sqrt(sum(x*x for x in vec))
7         v.append([x/mag for x in vec])
8
9     for _ in range(epochs):
```

```

10     new_v = []
11     for i in range(n):
12         grad = [0.0] * dim
13         for (u, val_v), weight in zip(edges, weights):
14             if u == i:
15                 for k in range(dim): grad[k] += weight * v[val_v][k]
16             elif val_v == i:
17                 for k in range(dim): grad[k] += weight * v[u][k]
18         updated = [v[i][k] - lr * grad[k] for k in range(dim)]
19         mag = math.sqrt(sum(x*x for x in updated))
20         new_v.append(v[i] if mag < 1e-9 else [x/mag for x in updated])
21     v = new_v
22     return v
23
24 def goemans_williamson_max_cut(n, edges, weights, vectors, trials=500):
25     """Applies randomized hyperplane rounding to partition vertices."""
26     dim = len(vectors[0])
27     best_weight, best_assignment = -1.0, []
28     for _ in range(trials):
29         r = [random.gauss(0.0, 1.0) for _ in range(dim)]
30         mag = math.sqrt(sum(x*x for x in r))
31         r = [x/mag for x in r]
32
33         assignment = [1 if sum(vectors[i][k] * r[k] for k in range(dim))
34                       >= 0 else 0 for i in range(n)]
35         weight = compute_cut_weight(assignment, edges, weights)
36         if weight > best_weight:
37             best_weight, best_assignment = weight, assignment
38     return best_assignment, best_weight

```

Listing 18.1: Goemans-Williamson Max-Cut Algorithm

18.4 Analysis: The 0.878 Performance Bound

We formulate the Max-Cut problem as a quadratic integer program: maximize $\frac{1}{2} \sum_{(i,j) \in E} w_{ij}(1 - y_i y_j)$ s.t. $y_i \in \{-1, 1\}$. By relaxing the scalar variables $y_i \in \{-1, 1\}$ to unit vectors $v_i \in \mathbb{R}^n$, we obtain the Semidefinite Programming relaxation:

$$\text{Maximize } \frac{1}{2} \sum_{(i,j) \in E} w_{ij}(1 - v_i \cdot v_j) \quad \text{s.t.} \quad v_i \cdot v_i = 1 \quad (\forall i \in V)$$

Under Goemans-Williamson randomized rounding, we choose a random unit vector r and partition based on $\text{sign}(v_i \cdot r)$. The probability that edge (i, j) is cut (i.e. v_i, v_j are separated by the random hyperplane) is exactly θ_{ij}/π , where $\theta_{ij} = \arccos(v_i \cdot v_j)$ is the angle between v_i and v_j . By linearity of expectation, the expected weight of the cut is:

$$E[\text{Weight}] = \sum_{(i,j) \in E} w_{ij} \frac{\theta_{ij}}{\pi}$$

Comparing this term-by-term with the SDP relaxation's objective, we bound the ratio:

$$\alpha = \min_{\theta \in [0, \pi]} \frac{\theta/\pi}{(1 - \cos \theta)/2} \approx 0.87856$$

Thus, the expected cut weight is at least $0.878 \cdot \text{OPT}_{SDP} \geq 0.878 \cdot \text{OPT}$.

18.5 Applications

- **Statistical Physics (Ising Spin Glasses):** Finding the minimum energy configuration (ground state) of spin glass magnetic systems.
- **VLSI Layout and Cross-Talk Minimization:** Splitting circuit gates into two layers to minimize inter-layer connection crossovers.
- **Image Segmentation:** Clustering pixels into foreground and background by maximizing cut weight along contrast boundaries.

Real Example: VLSI Circuit Cross-Talk Minimization

Scenario: A chip designer (e.g. Intel) layouts $n = 5,000$ transistors on a chip. Certain pairs of wires have high electrical interference (cross-talk) if routed on the same layer. The designer wants to partition transistors into two separate routing layers to minimize same-layer cross-talk. **Model:** Max-Cut. Transistors are vertices, cross-talk interferences are edge weights. Layer division is the cut. The Goemans-Williamson algorithm partitions the circuit layers. **Why SDP matters:** Deterministic algorithms easily get stuck in local minima on highly interconnected networks. The Goemans-Williamson SDP algorithm embeds the netlist geometrically, placing highly interfering nodes on opposite sides of a sphere, and partitions them globally with a guaranteed 87.8% efficiency.

18.6 Tight Example: The 5-Cycle Graph

Consider the 5-cycle graph C_5 with unit edge weights. The optimal Max-Cut value is 4 (achieved by partitioning into sets of size 2 and 3). The optimal 2D embedding of C_5 places the vectors on a circle with angle $\theta = 4\pi/5 = 144^\circ$ between adjacent nodes. The SDP objective value is:

$$\text{OPT}_{SDP} = 5 \cdot \frac{1 - \cos(144^\circ)}{2} \approx 5 \cdot 0.9045 = 4.5225$$

The expected cut weight under random hyperplane rounding is:

$$E[\text{Weight}] = 5 \cdot \frac{144^\circ}{180^\circ} = 4.0000$$

The ratio of the expected cut weight to the SDP optimal value is $4.0/4.5225 \approx 0.8844$, which is extremely close to the theoretical bound of 0.878.

18.7 Exercises

Exercise 18.1. Prove that the probability that two unit vectors v_i, v_j are separated by a random hyperplane r is exactly θ_{ij}/π , where $\theta_{ij} = \arccos(v_i \cdot v_j)$.

Exercise 18.2. Show that for a triangle graph K_3 with unit weights, the optimal Max-Cut is 2, while the SDP relaxation value is 2.25. What is the ratio?

Chapter 19

Multiway Cut LP Rounding

19.1 Intuition: Simplex-Distance LP and CKR Rounding

Intuition 19.1. Multiway Cut LP Rounding: For the Multiway Cut problem, the standard isolating cuts algorithm yields a $2(1-1/k)$ -approximation. However, by formulating the problem as a Simplex-distance LP relaxation (where vertices are embedded in a simplex and edge costs correspond to distances on this simplex), we can achieve a better approximation ratio of 1.5.

CKR Rounding: The Călinescu-Karloff-Rabani (CKR) randomized rounding scheme selects a random permutation of terminals and a random radius $r \in (0, 1/2)$, assigning each vertex to the first terminal in the permutation whose distance is within the radius. This global randomized routing scheme cuts edges with probability bounded by 1.5 times their LP contribution.

19.2 Problem Definition

Given an undirected graph $G = (V, E)$ with non-negative edge costs $c_e \geq 0$, and a set of k terminals $S = \{s_1, \dots, s_k\} \subseteq V$, the *Multiway Cut* problem is to find a minimum cost subset of edges $C \subseteq E$ whose removal disconnects every terminal from all other terminals.

The problem is NP-hard. We seek a randomized 1.5-approximation using the Simplex-distance LP relaxation and CKR randomized rounding.

19.3 Algorithm and Python Implementation

We solve the LP relaxation by formulating the primal-dual problem and then apply the CKR randomized rounding.

```
1 def calinescu_karloff_rabani_rounding(  
2     n, edges, costs, terminals, d, trials=500  
3 ):  
4     """Applies CKR randomized rounding to multiway cut LP solution."""  
5     k = len(terminals)  
6     best_cost = float('inf')  
7     best_cut = []  
8
```

```

9   for _ in range(trials):
10      perm = list(range(k))
11      random.shuffle(perm)
12      r = random.uniform(0.0, 0.5)
13
14      assignment = [-1] * n
15      for v in range(n):
16          assigned = False
17          for idx in perm:
18              if d[v][idx] > r:
19                  assignment[v] = idx
20                  assigned = True
21                  break
22          if not assigned:
23              assignment[v] = perm[-1]
24
25      cut = []
26      cost = 0.0
27      for e, (u, v) in enumerate(edges):
28          if assignment[u] != assignment[v]:
29              cut.append((u, v))
30              cost += costs[e]
31
32      if cost < best_cost:
33          best_cost, best_cut = cost, cut
34
35      return best_cut, best_cost

```

Listing 19.1: CKR Randomized Rounding for Multiway Cut

19.4 Analysis: Simplex Embedding and 1.5 Bound

In the Simplex-distance LP relaxation, we associate a vector $d(v) = (d_1(v), \dots, d_k(v))$ with each vertex v representing its location in the k -dimensional simplex. The distance between u and v is measured as $\frac{1}{2} \sum_{i=1}^k |d_i(u) - d_i(v)|$. The LP is formulated as:

$$\min \frac{1}{2} \sum_{(u,v) \in E} c_{uv} \sum_{i=1}^k |d_i(u) - d_i(v)|$$

subject to:

$$\sum_{i=1}^k d_i(v) = 1 \quad (\forall v \in V), \quad d_i(s_j) = 1 \text{ if } i = j \text{ else } 0, \quad d_i(v) \geq 0 \quad (\forall v, i)$$

Under the CKR randomized rounding scheme, the probability that edge $e = (u, v)$ is cut is bounded by:

$$\Pr[\text{edge } (u, v) \text{ is cut}] \leq 1.5 \cdot \left(\frac{1}{2} \sum_{i=1}^k |d_i(u) - d_i(v)| \right)$$

By linearity of expectation, the expected cost of the cut is at most $1.5 \cdot \text{OPT}_{LP} \leq 1.5 \cdot \text{OPT}$.

19.5 Applications

- **Distributed Database Partitioning:** Dividing data tables across k database servers to minimize the cost of cross-server transactional joins.
- **Image Segmentation with Multiple Labels:** Segmenting an image into k distinct semantic regions (e.g. road, sky, car) while minimizing boundary mismatch costs.
- **Network Domain Isolation:** Separating k secure networks sharing a common backbone using firewall boundaries.

Real Example: Image Segmentation with Multiple Labels

Scenario: A self-driving car's vision system (e.g. Waymo) segments an HD camera frame into $k = 3$ categories: Road, Obstacles, and Background. Pixels represent vertices, and adjacent pixels are connected by edges with weight proportional to color similarity. **Model:** Multiway Cut on a grid graph, where the terminals represent seeds for each category. **Why CKR LP Rounding matters:** Isolating cuts can yield asymmetric boundary segments that cross high-contrast edges. The CKR Simplex LP embedding places pixel classes smoothly on a probability simplex, and the random permutation and radius ensure globally consistent cuts. This results in segment boundaries that align naturally with physical object edges.

19.6 Tight Example: The Spokes Graph

Consider a graph of 6 vertices: 3 terminals 0, 2, 4, a central vertex 5, and outer cycle vertices. The LP relaxation allows the central vertex to be embedded at $(1/3, 1/3, 1/3)$, yielding an LP value of 12.0. **CKR Rounding Results:** With high probability, the cut total cost is 12.0, which matches the optimal integral cut value of 12.0.

19.7 Exercises

Exercise 19.1. *Prove that the Simplex-distance LP relaxation for $k = 3$ terminals has an integrality gap of at most $12/11$.*

Exercise 19.2. *Implement a deterministic version of the CKR rounding by trying all possible permutations and picking the best threshold.*

Chapter 21

Steiner Forest

21.1 Intuition: Connective Components and Reverse Deletion

Intuition 21.1. *Peeling and Layering Intuition:* Steiner Forest generalizes Steiner Tree by requiring only specific pairs of terminals (s_i, t_i) to be connected. In the primal-dual schema, we raise dual variables y_C associated with active connected components (components separating at least one terminal pair). When an edge becomes tight, it is added to the forest. This can cause redundant connections. The crucial step is a reverse-delete phase: we examine chosen edges in reverse order and delete any edge that is not necessary to maintain the connection of all terminal pairs.

21.2 Problem Definition

Given an undirected graph $G = (V, E)$ with edge costs $c_e \geq 0$, and a set of k terminal pairs $S = \{(s_1, t_1), \dots, (s_k, t_k)\}$, the *Steiner Forest* problem is to find a minimum cost subgraph $F \subseteq E$ such that s_i and t_i are in the same connected component of (V, F) for every $i \in \{1, \dots, k\}$.

The problem is NP-hard. It admits a 2-approximation using the Agrawal-Klein-Ravi (AKR) primal-dual algorithm.

21.3 Algorithm and Python Implementation

We implement the AKR primal-dual schema: growing dual variables for active components, adding tight edges to the forest, and performing reverse-delete pruning.

```
1 def steiner_forest_primal_dual(vertices, edges, pairs):
2     """AKR Primal-Dual 2-approximation for Steiner Forest."""
3     chosen_edges = []
4     L = {i: 0.0 for i in range(len(edges))} # Dual values on edges
5
6     while True:
7         components = find_components(vertices, chosen_edges)
8         node_to_comp = {node: idx for idx, comp in enumerate(components)
9                         for node in comp}
```

```

10      # Identify active components
11      active = set()
12      for s, t in pairs:
13          if node_to_comp[s] != node_to_comp[t]:
14              active.add(node_to_comp[s])
15              active.add(node_to_comp[t])
16
17      if not active: break
18
19      # Find next tight edge
20      best_delta, best_edge_indices = float('inf'), []
21      edge_rates = []
22      for i, (u, v, cost) in enumerate(edges):
23          if (u, v) in chosen_edges or (v, u) in chosen_edges:
24              edge_rates.append(0); continue
25          c_u, c_v = node_to_comp[u], node_to_comp[v]
26          if c_u == c_v:
27              edge_rates.append(0); continue
28          rate = (1 if c_u in active else 0) + (1 if c_v in active else
29              0)
29          edge_rates.append(rate)
30          if rate > 0:
31              delta = (cost - L[i]) / rate
32              if delta < best_delta:
33                  best_delta, best_edge_indices = delta, [i]
34              elif abs(delta - best_delta) < 1e-9:
35                  best_edge_indices.append(i)
36
37      for i in range(len(edges)):
38          if edge_rates[i] > 0: L[i] += best_delta * edge_rates[i]
39      for idx in best_edge_indices:
40          u, v, _ = edges[idx]
41          chosen_edges.append((u, v))
42
43      # Reverse-delete pruning
44      pruned = list(chosen_edges)
45      for edge in reversed(chosen_edges):
46          pruned.remove(edge)
47          if not all(is_connected(vertices, pruned, s, t) for s, t in pairs)
48              :
49              pruned.append(edge)
50      return pruned

```

Listing 21.1: Primal-Dual Steiner Forest Algorithm

21.4 Analysis: Factor 2 Approximation

The LP relaxation of the Steiner Forest problem is:

$$\min \sum_{e \in E} c_e x_e \quad \text{s.t.} \quad \sum_{e \in \delta(U)} x_e \geq 1 \quad (\forall U \in \mathcal{A}), \quad x_e \geq 0$$

where $\mathcal{A}$ is the set of all cuts separating some terminal pair. The dual LP is:

$$\max \sum_{U \in \mathcal{A}} y_U \quad \text{s.t.} \quad \sum_{U: e \in \delta(U)} y_U \leq c_e \quad (\forall e \in E), \quad y_U \geq 0$$

For any edge e in the final pruned forest F , the dual constraint is tight: $\sum_{U: e \in \delta(U)} y_U = c_e$. Thus, the cost of the forest is:

$$\sum_{e \in F} c_e = \sum_{e \in F} \sum_{U: e \in \delta(U)} y_U = \sum_{U \in \mathcal{A}} y_U \cdot \deg_F(U)$$

where $\deg_F(U)$ is the number of edges of F crossing the cut $\delta(U)$. To prove a 2-approximation, it suffices to show that at any step:

$$\sum_{U \text{ active}} \deg_F(U) \leq 2 \cdot |\{U \text{ active}\}|$$

This holds because if we contract each component of $G = (V, F)$ to a single node, the active components form a set of nodes in a forest where every leaf node is active, and no node of degree 1 is inactive (due to reverse-delete pruning). The average degree of active nodes in such a forest is at most 2. Thus, $\sum_{e \in F} c_e \leq 2 \sum y_U \leq 2 \cdot \text{OPT}$.

21.5 Applications

- **Telecommunication Network Design:** Laying cables to connect specific pairs of client servers while sharing as much infrastructure as possible.
- **VLSI Routing:** Connecting pairs of terminals on an integrated circuit layout.
- **Highways Construction:** Planning transport pathways that must connect disjoint cities of interest.

Real Example: Multi-Tenant Virtual Private Network Routing

Scenario: A network provider (e.g. Equinix) needs to establish secure connections for multiple enterprise tenants in a shared data center network. Tenant A needs to connect location s_1 to t_1 . Tenant B needs to connect s_2 to t_2 . The provider pays a cost for each physical link activated and wants to minimize the total link cost. **Model:** Steiner Forest. Each tenant represents a terminal pair. Activated links form the forest. The AKR primal-dual algorithm finds the cost-minimizing shared routing tree/forest. **Why Primal-Dual matters:** Connecting tenants individually leads to redundant duplicate links (e.g. each running its own Steiner Tree), which is highly suboptimal. Steiner Forest optimizes the entire topology globally. The primal-dual schema runs in polynomial time and guarantees a cost within a factor of 2 of the absolute optimum, even for huge transit networks.

21.6 Tight Example: The Redundant Cycle

Consider a cycle of three nodes 0, 1, 2 with edges (0, 1) of cost 1, (1, 2) of cost 2, and (2, 0) of cost 1. We want to connect the terminal pair (0, 2). The optimal solution is to select (2, 0) with cost 1. If we run primal-dual, the active components are $\{0\}$ and $\{2\}$. Edge slacks decrease. Edge

$(0, 1)$ and $(2, 0)$ both have rate 1 and cost 1. They become tight simultaneously and are added to F . The components merge into $\{0, 1, 2\}$, which connects 0 and 2. The set of chosen edges is $F = \{(0, 1), (2, 0)\}$. Now we run reverse delete: If we try deleting $(2, 0)$, 0 and 2 are not connected, so we keep it. If we try deleting $(0, 1)$, 0 and 2 are not connected, so we keep it. The final forest has edges $(0, 1)$ and $(2, 0)$ with total cost $1 + 1 = 2$. The optimal cost is 1 (the single edge $(2, 0)$ of cost 1). The approximation ratio is $2/1 = 2$. This matches the theoretical factor of 2.

21.7 Exercises

Exercise 21.1. *Construct an example showing that without the reverse-delete phase, the primal-dual algorithm can perform arbitrarily worse than a 2-approximation.*

Exercise 21.2. *Prove that the Steiner Forest LP relaxation has an integrality gap of exactly 2.*

Chapter 26

Max 2-SAT

26.1 Intuition: Anchoring with a True Variable

Intuition 26.1. Peeling and Layering Intuition: *Max 2-SAT generalizes Max-SAT by restricting every clause to contain at most 2 literals. While Goemans-Williamson achieves a 0.878 ratio for Max-Cut, we can apply Semidefinite Programming (SDP) to Max 2-SAT by introducing a special anchor vector v_0 representing the value True. For each variable x_i , we associate a unit vector v_i . Literals x_i and $\neg x_i$ are represented by v_i and $-v_i$. Under randomized rounding, we partition based on whether a vector v_i falls on the same side of a random hyperplane as the anchor vector v_0 . This yields a guaranteed 0.878 approximation, which can be further improved to 0.940 using LLZ rounding.*

26.2 Problem Definition

Given n boolean variables $x_1, \dots, x_n$ and m clauses $C_1, \dots, C_m$, each containing at most 2 literals, with weights $w_j \geq 0$, the *Maximum 2-Satisfiability* (Max 2-SAT) problem is to find a truth assignment to the variables that maximizes the total weight of satisfied clauses.

The problem is NP-hard. We seek a randomized 0.878-approximation using SDP.

26.3 Algorithm and Python Implementation

We solve the Max 2-SAT SDP relaxation by optimizing vectors $v_0, v_1, \dots, v_n$ on the unit sphere and rounding using sign matching against v_0 .

```
1 def optimize_max_2sat_vectors(n_vars, clauses, weights, dim=8, lr=0.05,
2   epochs=300):
3     """Optimizes vectors v_0, v_1, ..., v_n on the unit sphere in R^dim."""
4     n_vectors = n_vars + 1
5     v = []
6     for _ in range(n_vectors):
7         vec = [random.gauss(0.0, 1.0) for _ in range(dim)]
8         mag = math.sqrt(sum(x*x for x in vec))
9         v.append([x/mag for x in vec])
```

```

9
10 for _ in range(epochs):
11     new_v = []
12     for i in range(n_vectors):
13         grad = [0.0] * dim
14         for j, (l1, l2) in enumerate(clauses):
15             w = weights[j]
16             s1 = 1 if l1 > 0 else -1
17             v1_idx = abs(l1)
18             if l2 == 0:
19                 if i == 0:
20                     for k in range(dim): grad[k] += 0.5 * w * s1 * v[
21                         v1_idx][k]
22                 elif i == v1_idx:
23                     for k in range(dim): grad[k] += 0.5 * w * s1 * v
24                         [0][k]
25             else:
26                 s2 = 1 if l2 > 0 else -1
27                 v2_idx = abs(l2)
28                 if i == 0:
29                     for k in range(dim): grad[k] += 0.25 * w * (s1 * v
30                         [v1_idx][k] + s2 * v[v2_idx][k])
31                 elif i == v1_idx:
32                     for k in range(dim): grad[k] += 0.25 * w * (s1 * v
33                         [0][k] - s1 * s2 * v[v2_idx][k])
34                 elif i == v2_idx:
35                     for k in range(dim): grad[k] += 0.25 * w * (s2 * v
36                         [0][k] - s1 * s2 * v[v1_idx][k])
37             updated = [v[i][k] + lr * grad[k] for k in range(dim)]
38             mag = math.sqrt(sum(x*x for x in updated))
39             new_v.append(v[i] if mag < 1e-9 else [x/mag for x in updated])
40     v = new_v
41     return v

```

Listing 26.1: SDP Max 2-SAT Algorithm

26.4 Analysis: The SDP Relaxation and LLZ Rounding

We map each variable x_i to a unit vector $v_i \in \mathbb{R}^{n+1}$ and introduce a unit vector v_0 representing True. The literal l has vector $v(l) = v_i$ if $l = x_i$, and $v(l) = -v_i$ if $l = \neg x_i$. The truth value of literal l is relaxed to $\frac{1+v(l) \cdot v_0}{2}$. For a clause $l_1 \vee l_2$, its satisfaction probability is relaxed to:

$$\text{val}(l_1 \vee l_2) = \frac{3 + v(l_1) \cdot v_0 + v(l_2) \cdot v_0 - v(l_1) \cdot v(l_2)}{4}$$

The SDP relaxation is:

$$\text{Maximize} \quad \sum_{C_j=l_1 \vee l_2} w_j \text{val}(C_j) \quad \text{s.t.} \quad v_i \cdot v_i = 1 \quad (\forall i \in \{0, \dots, n\})$$

Under randomized rounding, we choose a random unit vector $r \in \mathbb{R}^{n+1}$. Variable x_i is set to True if $\text{sign}(v_i \cdot r) = \text{sign}(v_0 \cdot r)$, and False otherwise. Goemans and Williamson proved that the probability

that clause $l_1 \vee l_2$ is satisfied is at least 0.878 times its relaxed SDP value. Lewin, Livnat, and Zwick (LLZ 2002) improved this by using a non-linear rotation function on the angles before hyperplane rounding, achieving a 0.940-approximation.

26.5 Applications

- **Logic Circuit Optimization:** Maximizing the number of gate checks satisfied in automated hardware testing.
- **Probabilistic Reasoning (Markov Logic Networks):** Finding the most probable explanation (MAP state) for a set of weighted boolean rules.
- **Social Network Balance:** Finding stable divisions of communities containing positive (friendship) and negative (hostility) links.

Real Example: Hardware Logic Testing and Verification

Scenario: An ASIC designer (e.g. AMD) verifies a circuit block. Due to pin constraints, we can only verify a subset of logic gates simultaneously. Each test case checks 2-literal logic clauses. We want to select a test vector maximizing checked gates. **Model:** Max 2-SAT. Checked gates are clauses. Goemans-Williamson SDP algorithm generates the test vectors. **Why SDP matters:** Circuits are highly dense networks. Simple greedy strategies fail to find configurations that satisfy a high percentage of gates. SDP provides a global geometric optimization, ensuring test coverage is within a small fraction of the absolute mathematical maximum.

26.6 Tight Example: The All-Combinations Cycle

Consider 2 variables x_1, x_2 and 4 clauses containing all possible sign combinations: $(x_1 \vee x_2)$, $(\neg x_1 \vee x_2)$, $(x_1 \vee \neg x_2)$, and $(\neg x_1 \vee \neg x_2)$. No boolean assignment can satisfy all 4 clauses, so the optimal Max 2-SAT weight is 3. The SDP relaxation optimizes vectors v_1, v_2 and v_0 . The objective value is $\sum_{j=0}^3 \frac{3 \pm v_1 \cdot v_0 \pm v_2 \cdot v_0 - v_1 \cdot v_2}{4} = 3 - v_1 \cdot v_2$. By placing v_1, v_2 orthogonal to each other ($v_1 \cdot v_2 = 0$) and setting their dot products with v_0 to maximize the sum, the SDP achieves a value of 3. Randomized hyperplane rounding splits the vectors, ensuring exactly 3 out of 4 clauses are satisfied. The ratio is $3/3 = 1.0$, showing perfect behavior on this symmetric instance.

26.7 Exercises

Exercise 26.1. Show that if we assign x_i to True with probability $(1 + v_i \cdot v_0)/2$, the expected number of satisfied clauses for single-literal clauses matches the LP/SDP value exactly.

Exercise 26.2. Construct a Max 2-SAT instance with 3 variables and 6 clauses where the optimal assignment satisfies 5 clauses, and trace the vector updates.

Chapter 22

Steiner Network

22.1 Intuition: LP Extreme Points and Iterative Rounding

Intuition 22.1. Peeling and Layering Intuition: Primal-dual schemas and randomized rounding are powerful, but some problems require a more direct structural exploitation of the LP relaxation. Kamal Jain (1998) introduced Iterative Rounding for the Survivable Network Design Problem (SNDP). The key insight is that any extreme point (basic feasible solution) of the SNDP LP relaxation must contain at least one edge e with fractional value $x_e^* \geq 1/2$. By rounding this edge up to 1, reducing the connectivity requirements, and solving the LP on the remaining graph iteratively, we obtain a guaranteed 2-approximation.

22.2 Problem Definition

Given an undirected graph $G = (V, E)$ with edge costs $c_e \geq 0$, and connectivity requirements $r_{uv} \geq 0$ for each pair of vertices $u, v \in V$, the *Steiner Network* (or Survivable Network Design) problem is to find a minimum cost subgraph $H \subseteq E$ containing at least r_{uv} edge-disjoint paths between u and v for all $u, v \in V$.

The problem is NP-hard. We seek a 2-approximation using Jain's iterative rounding algorithm.

22.3 Algorithm and Python Implementation

We solve the LP relaxation of SNDP by formulating its dual (which has a naturally feasible origin since edge costs are non-negative) and iteratively round edges with $x_e \geq 0.5$.

```
1 def solve_sndp_lp_phase(n, edges, costs, r, fixed_edges):
2     """Solves the continuous LP relaxation for SNDP under current
3     requirements."""
4     cuts = get_all_cuts(n)
5     n_cuts, n_edges = len(cuts), len(edges)
6     active_edges = [i for i in range(n_edges) if i not in fixed_edges]
7     n_active = len(active_edges)
8
9     # Calculate requirements f(S) for each cut S
10    f = []
```

```

10     for cut in cuts:
11         val = max([req for (u, v), req in r.items() if (u in cut) != (v in
12             cut)] + [0])
13         fixed_crossing = sum(1 for idx in fixed_edges if (edges[idx][0] in
14             cut) != (edges[idx][1] in cut))
15         f.append(max(0, val - fixed_crossing))
16
17     n_cols = n_cuts + n_active
18     A_dual, b_dual = [], []
19     for j, edge_idx in enumerate(active_edges):
20         u, v = edges[edge_idx]
21         row = [0.0] * n_cols
22         for s_idx, cut in enumerate(cuts):
23             if (u in cut) != (v in cut): row[s_idx] = 1.0
24         row[n_cuts + j] = -1.0
25         A_dual.append(row)
26         b_dual.append(costs[edge_idx])
27
28     c_dual = [float(val) for val in f] + [-1.0] * n_active
29     solver = Simplex(A_dual, b_dual, c_dual)
30     dual_sol, dual_obj = solver.solve()
31
32     final_obj = solver.obj_row
33     x = [0.0] * n_edges
34     for idx in fixed_edges: x[idx] = 1.0
35     for j, edge_idx in enumerate(active_edges):
36         x[edge_idx] = max(0.0, final_obj[n_cols + j])
37     return x

```

Listing 22.1: Jain's Iterative Rounding Algorithm

22.4 Analysis: Jain's Theorem and Factor 2

The LP relaxation for the Steiner Network problem is:

$$\min \sum_{e \in E} c_e x_e \quad \text{s.t.} \quad \sum_{e \in \delta(S)} x_e \geq f(S) \quad (\forall S \subset V), \quad 0 \leq x_e \leq 1$$

where $f(S) = \max_{u \in S, v \notin S} r_{uv}$. An extreme point (or basic feasible solution) of this LP is defined by a set of linearly independent tight cut constraints. Jain proved a fundamental structural property of these extreme points:

Theorem 22.1 (Jain 1998). *For any connectivity requirement function f that is weakly supermodular, any basic feasible solution to the LP relaxation has at least one edge e with $x_e \geq 1/2$.*

Based on this theorem, we round x_e to 1, which increases its cost to $c_e \leq 2 \cdot c_e x_e$. By induction on the iterations, the total cost of the rounded edges is bounded by:

$$\sum_{e \in H} c_e \leq 2 \cdot \text{OPT}_{LP} \leq 2 \cdot \text{OPT}$$

which proves that Jain's iterative rounding is a 2-approximation.

22.5 Applications

- **Survivable Network Design:** Laying communication links that can survive up to k simultaneous link failures by maintaining k disjoint paths between key nodes.
- **Power Grid Design:** Constructing high-capacity power lines that ensure alternative routes during generator outages.
- **Disaster Recovery Routing:** Deploying redundant emergency networks connecting key rescue centers.

Real Example: High-Availability Telecommunication Backbones

Scenario: A telecom operator (e.g. AT&T) connects regional routing hubs. Certain hubs (e.g., Chicago and New York) require a high-availability channel with 3 edge-disjoint paths to survive up to 2 simultaneous fiber cuts. Other hubs only require 1 path. **Model:** Steiner Network. Hubs are vertices, connectivity requirements are $r_{uv} \in \{0, 1, 3\}$. Iterative rounding finds the optimal network. **Why Iterative Rounding matters:** Dual-based schemas or randomized rounding cannot handle high disjointness constraints ($r_{uv} \geq 3$) efficiently. Jain's iterative rounding algorithm handles general integer demands directly, ensuring a cost-efficient layout with a strict mathematical guarantee of being within $2\times$ of the global optimum.

22.6 Tight Example: The Multi-Path Cycle

Consider a 4-node graph with edges $(0, 1), (1, 2), (2, 3), (3, 0)$ of costs 2, 3, 2, 3, and a diagonal edge $(0, 2)$ of cost 4. Let the requirements be $r(0, 2) = 2$ and $r(1, 3) = 1$. The optimal integer solution is to select edges $(0, 1), (1, 2), (2, 3), (3, 0)$ with total cost $2 + 3 + 2 + 3 = 10$. Our iterative rounding algorithm solves the LP, rounds the diagonal edge $(0, 2)$ (since its LP value is ≥ 0.5), and then rounds $(0, 1), (1, 2), (2, 3)$, yielding a total cost of $2 + 3 + 2 + 4 = 11$. The approximation ratio is $11/10 = 1.1$, which is well within the factor 2 performance bound.

22.7 Exercises

Exercise 22.2. *Prove that if all connectivity requirements r_{uv} are at most 1, the Steiner Network problem is equivalent to the Steiner Forest problem.*

Exercise 22.3. *Construct a basic feasible solution for a triangle graph with connectivity requirements 1, and show that at least one edge has $x_e \geq 1/2$.*

Chapter 23

Feedback Vertex Set via Primal-Dual

23.1 Intuition: Cycle Hitting and Vertex Budgets

Intuition 23.1. Peeling and Layering Intuition: The Feedback Vertex Set (FVS) problem can be viewed as a Hitting Set problem where we want to select a minimum weight set of vertices that intersects every cycle. In the primal-dual framework, we grow dual variables y_C associated with cycles. To achieve a 2-approximation, we grow duals on cycles in the remaining graph, but we scale the dual pressure rate on vertex v by $d(v) - 1$, where $d(v)$ is its degree. When a vertex's weight constraint is tight, it is added to the feedback set. Finally, we prune redundant vertices in reverse order of addition.

23.2 Problem Definition

Given an undirected graph $G = (V, E)$ with non-negative vertex weights $w_v \geq 0$, the *Feedback Vertex Set* (FVS) problem is to find a minimum weight subset of vertices $F \subseteq V$ such that $G \setminus F$ is acyclic (a forest).

The problem is NP-hard. It admits a 2-approximation using the Bafna-Berman-Fujito primal-dual algorithm.

23.3 Algorithm and Python Implementation

We implement the primal-dual algorithm: recursively pruning vertices of degree ≤ 1 , finding active cycles, growing duals at rate $d(v) - 1$, and running a reverse-delete phase.

```
1 def primal_dual_fvs(vertices, edges, weights):
2     """Bafna-Berman-Fujito primal-dual 2-approximation for undirected FVS.
3         """
4     w = dict(weights)
5     S = []
6     active_vertices = set(vertices)
7     active_edges = list(edges)
8
9     while True:
10         # Clean vertices of degree <= 1
```

```

10     while True:
11         deg = {v: 0 for v in active_vertices}
12         for u, v in active_edges:
13             deg[u] += 1; deg[v] += 1
14         to_remove = [v for v in active_vertices if deg[v] <= 1]
15         if not to_remove: break
16         for v in to_remove:
17             active_vertices.remove(v)
18             active_edges = [(u, x) for u, x in active_edges if u != v
19                             and x != v]
19
20         if not active_vertices: break
21         cycle = find_any_cycle(active_vertices, active_edges)
22         if not cycle: break
23
24         deg = {v: 0 for v in active_vertices}
25         for u, v in active_edges:
26             deg[u] += 1; deg[v] += 1
27
28         # Grow dual on cycle: rate is d(v) - 1
29         delta = float('inf')
30         for v in cycle:
31             rate = deg[v] - 1
32             if rate > 0: delta = min(delta, w[v] / rate)
33
34         # Update weights and find tight vertex
35         tight_vertex = None
36         for v in cycle:
37             rate = deg[v] - 1
38             w[v] -= delta * rate
39             if w[v] <= 1e-9 and tight_vertex is None:
40                 tight_vertex = v
41         if tight_vertex is None: tight_vertex = cycle[0]
42
43         S.append(tight_vertex)
44         active_vertices.remove(tight_vertex)
45         active_edges = [(u, v) for u, v in active_edges if u !=
46                         tight_vertex and v != tight_vertex]
47
48         # Reverse-delete pruning
49         pruned = list(S)
50         for v in reversed(S):
51             pruned.remove(v)
52             if not is_acyclic(set(vertices), edges, set(pruned)):
53                 pruned.append(v)
54         return pruned

```

Listing 23.1: Primal-Dual Feedback Vertex Set

23.4 Analysis: Dual Pressure and Factor 2

The FVS problem can be relaxed to the following linear program:

$$\min \sum_{v \in V} w_v x_v \quad \text{s.t.} \quad \sum_{v \in C} x_v \geq 1 \quad (\forall \text{ cycles } C), \quad x_v \geq 0$$

The dual LP is:

$$\max \sum_C y_C \quad \text{s.t.} \quad \sum_{C: v \in C} y_C \leq w_v \quad (\forall v \in V), \quad y_C \geq 0$$

For any vertex v in the final pruned feedback set F , its dual constraint is tight: $\sum_{C: v \in C} y_C (d_C(v) - 1) = w_v$, where $d_C(v) - 1$ is the dual growth rate. The total cost of the FVS is:

$$\sum_{v \in F} w_v = \sum_{v \in F} \sum_{C: v \in C} y_C (d_C(v) - 1) = \sum_C y_C \sum_{v \in F \cap C} (d_C(v) - 1)$$

A clean combinatorial charging argument shows that for any cycle C chosen by the algorithm, after reverse-delete pruning, the remaining feedback set vertices in C satisfy:

$$\sum_{v \in F \cap C} (d_C(v) - 1) \leq 2$$

This holds because F is a minimal feedback vertex set, and the average degree of vertices in any minimal feedback set on a cycle is at most 2. Thus, the total cost is:

$$\sum_{v \in F} w_v \leq 2 \sum_C y_C \leq 2 \cdot \text{OPT}_{LP} \leq 2 \cdot \text{OPT}$$

which proves the 2-approximation factor.

23.5 Applications

- **Operating Systems and Databases Deadlock Resolution:** Identifying and terminating a minimum cost set of processes to break resource wait-for cycles.
- **VLSI Circuit Layout:** Breaking feedback loops in synchronous logic circuits by opening minimum gate nodes.
- **Hierarchical Social Networking:** Identifying influential users whose removal makes the relationship network hierarchical and free of opinion-echo loops.

Real Example: Operating System Deadlock Resolution

Scenario: A database server running on Microsoft SQL Server detects a set of deadlocks among transactional processes (represented as a wait-for cycle graph). Terminating transaction v incurs a cost w_v (proportional to rolled-back CPU work). We want to break all deadlocks with minimum rollback cost. **Model:** Feedback Vertex Set. Transactions are vertices, wait-for dependencies are edges. The primal-dual algorithm finds the minimum cost rollback set. **Why Primal-Dual matters:** Rollbacks must happen in real time. Running exact integer programs is too slow and can block database execution. The primal-dual schema runs in polynomial time, finds tight feedback sets instantly, and guarantees that the rolled-back work cost is at most twice the optimal minimum rollback cost.

23.6 Tight Example: Intersecting Loops

Consider a graph of vertices $0, 1, 2, 3, 4$ and two cycles $0 - 1 - 2 - 0$ and $2 - 3 - 4 - 2$ sharing vertex 2. Let the shared vertex 2 have weight $w_2 = 3.0$ and other vertices have weight 10.0. The optimal feedback set is $\{2\}$ with cost 3.0. If we run our primal-dual algorithm, it finds cycle $0 - 1 - 2 - 0$. The degrees are $d(0) = 2, d(1) = 2, d(2) = 4$. The rates are $d(v) - 1$, which gives rates 1, 1, 3. The maximum step is $\Delta = \min(10/1, 10/1, 3/3) = 1.0$. Vertex 2's weight decreases to $3 - 3 \times 1 = 0$, making it tight. We add 2 to S . Removing vertex 2 leaves the graph acyclic (with components $0 - 1$ and $3 - 4$), so the algorithm terminates. The final feedback set is $\{2\}$ with cost 3.0, matching the optimal cost.

23.7 Exercises

Exercise 23.1. *Prove that the Feedback Vertex Set problem is equivalent to the Hitting Set problem on the family of all cycles in the graph.*

Exercise 23.2. *Prove that the FVS LP relaxation has an integrality gap of exactly 2 on a simple cycle graph of length n with unit vertex weights.*

Chapter 24

Facility Location

24.1 Intuition: Primal-Dual with Filters

Intuition 24.1. Facility Location: We open facilities (pay f_i) and connect clients to nearest open facility (pay c_{ij}). The LP relaxation has facility variables y_i and connection variables x_{ij} .

Primal-Dual (Jain-Vazirani): Raise dual variables α_j for clients uniformly. When $\sum_j \alpha_j = f_i$ for some facility i , open it. When $\alpha_j = c_{ij}$ for some open facility, connect j to i . This gives a 3-approximation.

LP Rounding (Shmoys-Tardos-Aardal): Solve LP, filter clients with high connection cost, open facilities with $y_i \geq 1/2$, connect remaining clients. Also gives 3-approximation (improved to 1.5 for metric case by Byrka et al.).

24.2 Problem Definition

Definition 24.1 (Uncapacitated Facility Location). Given facilities F with opening costs f_i , clients C with connection costs c_{ij} , find $S \subseteq F$ minimizing:

$$\sum_{i \in S} f_i + \sum_{j \in C} \min_{i \in S} c_{ij}$$

24.3 Primal-Dual 3-Approximation

Algorithm 24.2 (Facility Location — Factor 3 (Primal-Dual)). 1. $\alpha_j = 0$ for all clients; all facilities closed

2. While unconnected clients exist:

- (a) Raise all α_j uniformly until some constraint becomes tight
- (b) If $\sum_{j \in \Gamma(i)} \alpha_j = f_i$, open facility i
- (c) If $\alpha_j = c_{ij}$ for open facility i , connect j to i

3. Output open facilities and connections

Theorem 24.3. This algorithm achieves a 3-approximation for Metric Facility Location.

24.3.1 Other Algorithms

Table 24.1: Facility Location Algorithms

Algorithm	Approximation	Technique
Jain-Vazirani Primal-Dual	3	Primal-Dual Schema
Shmoys-Tardos-Aardal LP Rounding	3	LP Rounding
Byrka et al. (metric)	1.5	LP + Filtering + Matching
Local Search (k-median)	$3 + \epsilon$	Local Search

Why not exact? Facility Location is NP-hard (reduction from Set Cover). The 1.5-approximation is current best for metric case; 1.463 is the lower bound (Guha-Khuller).

24.4 Python Implementation

```

1 def facility_location_primal_dual(facilities, clients):
2     """3-approx for Facility Location (Jain-Vazirani)."""
3     alpha = {j: 0.0 for j in clients}
4     open_facilities = set()
5     connected = set()
6     active = set(facilities.keys())
7
8     while len(connected) < len(clients):
9         # Find minimum delta to make constraint tight
10        min_delta = float('inf')
11        for i in active:
12            slack = facilities[i]['cost'] - sum(alpha[j] for j in clients
13                if j not in connected and j in facilities[i]['clients'])
14            if slack < min_delta:
15                min_delta = slack
16
17        for j in clients:
18            if j not in connected:
19                for i in active:
20                    if j in facilities[i]['clients']:
21                        slack = facilities[i]['clients'][j] - alpha[j]
22                        if slack < min_delta:
23                            min_delta = slack
24
25        if min_delta == float('inf'): break
26
27        # Raise all unconnected alphas
28        for j in clients:
29            if j not in connected:
30                alpha[j] += min_delta
31
32        # Open tight facilities
33        for i in list(active):
34            if sum(alpha[j] for j in clients

```

```

35         if j not in connected and j in facilities[i]['clients']
36             ])) >= facilities[i]['cost'] - 1e-9:
37             open_facilities.add(i)
38             active.remove(i)
39             for j in clients:
40                 if j not in connected and j in facilities[i]['clients']
41                     ]:
42                     connected.add(j)

return open_facilities

```

Listing 24.1: Facility Location Primal-Dual

24.5 Applications

- **Supply chain:** Warehouse placement for retail
- **Cloud computing:** Data center location
- **Telecommunications:** Cell tower placement
- **Emergency services:** Fire station / hospital location

Real Example: Amazon Fulfillment Center Placement

Scenario: Amazon wants to open $k = 5$ new fulfillment centers in the US to serve $n = 500$ zip-code demand regions. Each center has fixed opening cost f_i (land, construction) and per-unit shipping cost c_{ij} to region j . **Model:** Facilities = candidate locations (200 possible sites). Clients = 500 zip codes with demand d_j . Cost $f_i + \sum_j c_{ij} \cdot d_j$. Metric: c_{ij} = ground shipping cost (satisfies triangle inequality). **Why 3-approx matters:** If $\text{OPT} = \$100\text{M}/\text{year}$, our solution costs $\leq \$300\text{M}/\text{year}$. In practice, Jain-Vazirani primal-dual often achieves $< 1.3 \times \text{OPT}$. The guarantee ensures no "catastrophic" placement. **Real constraint:** Centers must serve minimum volume to be profitable. This is *capacitated facility location* — harder (no constant-factor unless f_i uniform). Practical approach: open extra centers to handle capacity, then merge nearby ones. **Alternative:** k -median (exactly k centers) — local search gives $3 + \epsilon$ approx. Used when budget fixes k but opening costs vary.

24.6 Exercises

Exercise 24.4. Show that the LP relaxation integrality gap for metric facility location is at least 1.463.

Exercise 24.5. Implement the k -median local search algorithm (swap open/closed facilities).

Chapter 30

Multicut in Trees

30.1 Intuition: LCA Depth Sorting and Dual Growth

Intuition 30.1. *Peeling and Layering Intuition:* The Multicut problem asks for a minimum cost set of edges whose removal disconnects given pairs of vertices. While NP-hard in general graphs, the problem in trees admits a simple and elegant primal-dual 2-approximation. We root the tree arbitrarily and compute the lowest common ancestor (LCA) for each pair. By processing the pairs in descending order of their LCA depth (from deepest to highest), we grow dual variables for uncut paths. When an edge becomes tight, we select the highest tight edge on the path (closest to the LCA) to maximize its overlap with other paths.

30.2 Problem Definition

Given a tree $T = (V, E)$ with non-negative edge costs $c_e \geq 0$, and a set of k demand pairs $S = \{(s_1, t_1), \dots, (s_k, t_k)\}$, the *Multicut in Trees* problem is to find a minimum cost subset of edges $C \subseteq E$ such that s_i and t_i are disconnected in $T \setminus C$ for all $i \in \{1, \dots, k\}$.

The problem is NP-hard. It admits a 2-approximation using the tree-based primal-dual algorithm.

30.3 Algorithm and Python Implementation

We root the tree at vertex 0, sort the pairs by LCA depth, grow dual variables for uncut paths, select the highest tight edge, and apply reverse-delete pruning.

```
1 def multicut_in_trees(n, edges, costs, pairs):
2     """Primal-dual 2-approximation for Multicut in Trees."""
3     parent, depth = get_tree_properties(n, edges)
4     pair_lcas = [(depth[get_lca(u, v, parent, depth)], i) for i, (u, v) in
5                   enumerate(pairs)]
6     pair_lcas.sort(key=lambda x: x[0], reverse=True)
7
8     edge_to_idx = {tuple(sorted(e)): idx for idx, e in enumerate(edges)}
9     load = [0.0] * len(edges)
10    chosen_edges = []
```

```

10
11 for _, pair_idx in pair_lcas:
12     u, v = pairs[pair_idx]
13     path_edges = get_path_edges(u, v, parent)
14     if any(e in chosen_edges for e in path_edges): continue
15
16     # Grow dual y_i for this pair
17     min_slack = float('inf')
18     best_edge = None
19     for e in path_edges:
20         e_idx = edge_to_idx[e]
21         slack = costs[e_idx] - load[e_idx]
22         if slack < min_slack:
23             min_slack, best_edge = slack, e
24         elif abs(slack - min_slack) < 1e-9:
25             # Break tie: choose higher edge (closer to root)
26             u1, v1 = e; u2, v2 = best_edge
27             if min(depth[u1], depth[v1]) < min(depth[u2], depth[v2]):
28                 best_edge = e
29
30     for e in path_edges:
31         load[edge_to_idx[e]] += min_slack
32     if best_edge is not None:
33         chosen_edges.append(best_edge)
34
35     # Reverse-delete pruning
36     pruned = list(chosen_edges)
37     for e in reversed(chosen_edges):
38         pruned.remove(e)
39         if not all(any(pe in pruned for pe in get_path_edges(u, v, parent))
40                   for u, v in pairs):
41             pruned.append(e)
42     return pruned

```

Listing 30.1: Primal-Dual Multicut in Trees

30.4 Analysis: Factor 2 Approximation

The LP relaxation of Multicut in Trees is:

$$\min \sum_{e \in E} c_e x_e \quad \text{s.t.} \quad \sum_{e \in P_i} x_e \geq 1 \quad (\forall i \in \{1, \dots, k\}), \quad x_e \geq 0$$

where P_i is the unique path between s_i and t_i . The dual LP is:

$$\max \sum_{i=1}^k y_i \quad \text{s.t.} \quad \sum_{i: e \in P_i} y_i \leq c_e \quad (\forall e \in E), \quad y_i \geq 0$$

For any edge e in the final pruned cut C , the dual constraint is tight: $\sum_{i: e \in P_i} y_i = c_e$. Thus, the cost of the cut is:

$$\sum_{e \in C} c_e = \sum_{e \in C} \sum_{i: e \in P_i} y_i = \sum_{i=1}^k y_i \cdot |C \cap P_i|$$

To prove a 2-approximation, we show that for any pair i where we grew a dual variable $y_i > 0$, the number of cut edges on its path satisfies $|C \cap P_i| \leq 2$. Since we sorted pairs by LCA depth and chose the highest tight edge on the path P_i (closest to $\text{lca}(i)$), any other cut edge on P_i must belong to some path P_j processed earlier, whose LCA is deeper. It can be shown that at most one such edge can lie on the branch from s_i to $\text{lca}(i)$, and at most one on the branch from t_i to $\text{lca}(i)$. Thus, the path P_i contains at most 2 edges of C . Therefore:

$$\sum_{e \in C} c_e \leq 2 \sum_{i=1}^k y_i \leq 2 \cdot \text{OPT}_{LP} \leq 2 \cdot \text{OPT}$$

which proves that the approximation ratio is exactly 2.

30.5 Applications

- **Network Subnet Isolation:** Placing firewalls on tree router links to isolate specific pairs of computers while minimizing link firewall costs.
- **Content Delivery Networks (CDNs):** Placing proxy servers on tree routing nodes to decouple client-server tree paths.
- **High-voltage Circuit Breakers:** Placing circuit breakers in tree-like electricity grids to isolate fault sources.

Real Example: Network Security Subnet Isolation

Scenario: An enterprise corporate network (e.g. Cisco) is structured as a tree. The security team wants to isolate specific pairs of sensitive subnets (s_i, t_i) (e.g., Finance and Guest Wi-Fi) by installing hardware firewalls on the links. Installing a firewall on link e costs c_e . **Model:** Multicut in Trees. Subnets are leaves, links are edges. The primal-dual algorithm finds the minimum cost link set to secure. **Why Primal-Dual matters:** Placing firewalls on every link is cost-prohibitive. Running integer programs is slow for huge corporate trees. The tree-based primal-dual algorithm runs in linear time and guarantees a security layout costing at most twice the absolute minimum firewall budget.

30.6 Tight Example: The Branching LCA

Consider a tree rooted at 0 with edges $(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)$ of costs 2, 4, 1, 3, 2, 2. Let the demand pairs be $(3, 4), (5, 6)$, and $(3, 5)$. The optimal solution is to cut edges $(1, 3)$ and $(2, 5)$, with total cost $1 + 2 = 3$. If we run our primal-dual algorithm: - Pairs are sorted by LCA depth: $(3, 4)$ (LCA 1, depth 1) and $(5, 6)$ (LCA 2, depth 1) are processed first, then $(3, 5)$ (LCA 0, depth 0). - Processing $(3, 4)$ yields dual $y_{(3,4)} = 1.0$ and tightens edge $(1, 3)$, which is added to C . - Processing $(5, 6)$ yields dual $y_{(5,6)} = 2.0$ and tightens edge $(2, 5)$, which is added to C . - Processing $(3, 5)$, the path is already cut by $(1, 3)$ and $(2, 5)$, so we skip it. - Pruning keeps both edges. The final cut is $\{(1, 3), (2, 5)\}$ with cost 3.0, matching the optimal cost.

30.7 Exercises

Exercise 30.1. Prove that the multicut problem on a path graph is exactly equivalent to the interval hitting set problem (where we want to select a minimum number of points to hit a set of intervals).

Exercise 30.2. *Prove that if the tree is a star graph (a center node connected to n leaf nodes), the Multicut in Trees problem is equivalent to the Vertex Cover problem.*

Bibliography

- [1] V. V. Vazirani, *Approximation Algorithms*, Springer, 2001.
- [2] N. Christofides, Worst-case analysis of a new heuristic for the travelling salesman problem, *Technical Report*, CMU, 1976.
- [3] U. Feige, A threshold of $\ln n$ for approximating set cover, *J. ACM*, 45(4):634–652, 1998.
- [4] K. Jain and V. Vazirani, Approximation algorithms for metric facility location and k-median problems using the primal-dual schema and Lagrangian relaxation, *J. ACM*, 48(2):274–296, 2001.
- [5] A. Karlin, N. Klein, S. Oveis Gharan, A (slightly) improved approximation algorithm for metric TSP, *STOC*, 2021.
- [6] PAAL Library — Practical Approximation Algorithms Library, <https://paal.mimuw.edu.pl>.